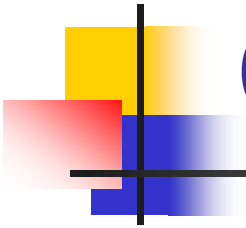


Introduction to Information Retrieval

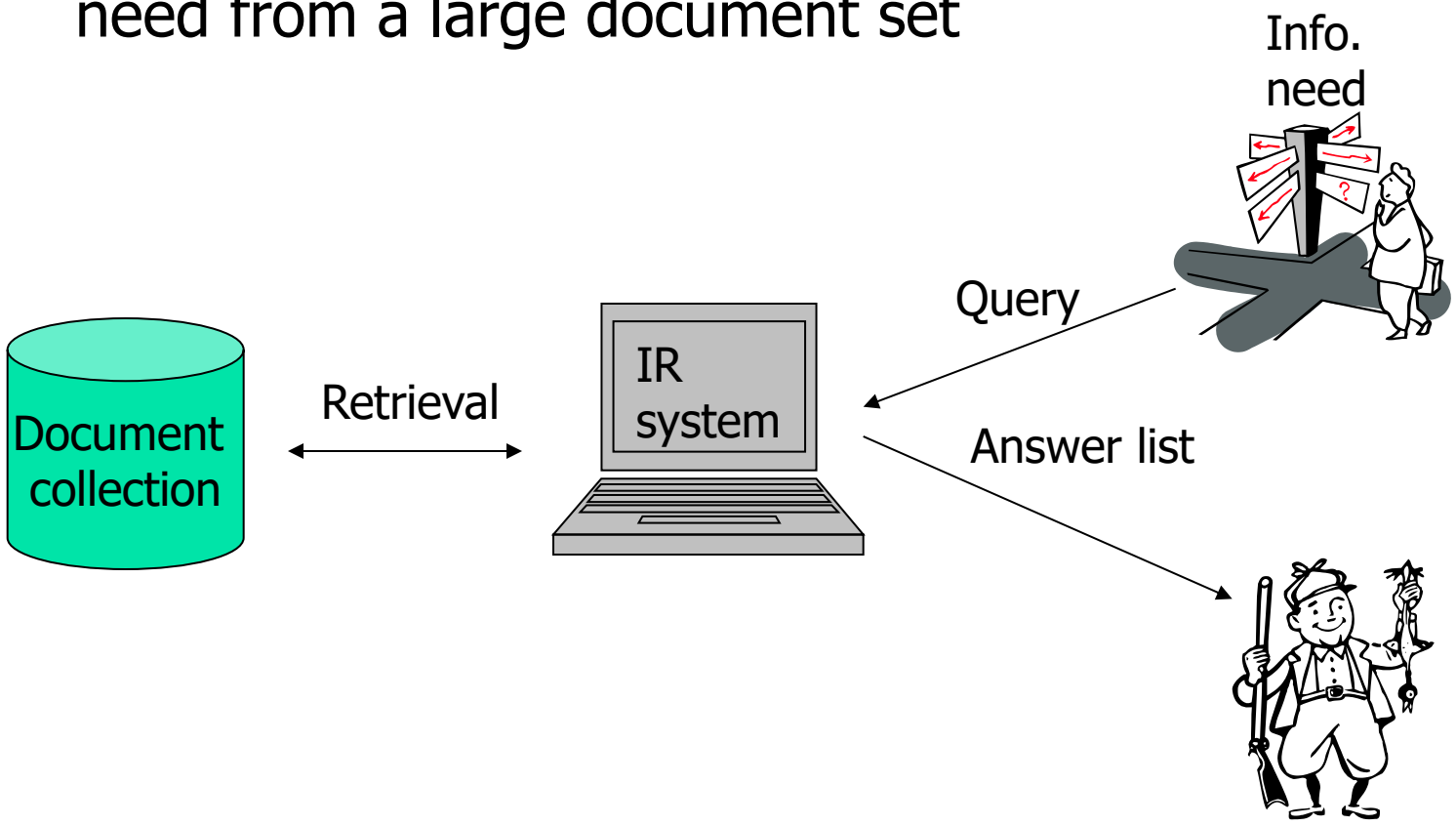


Outline

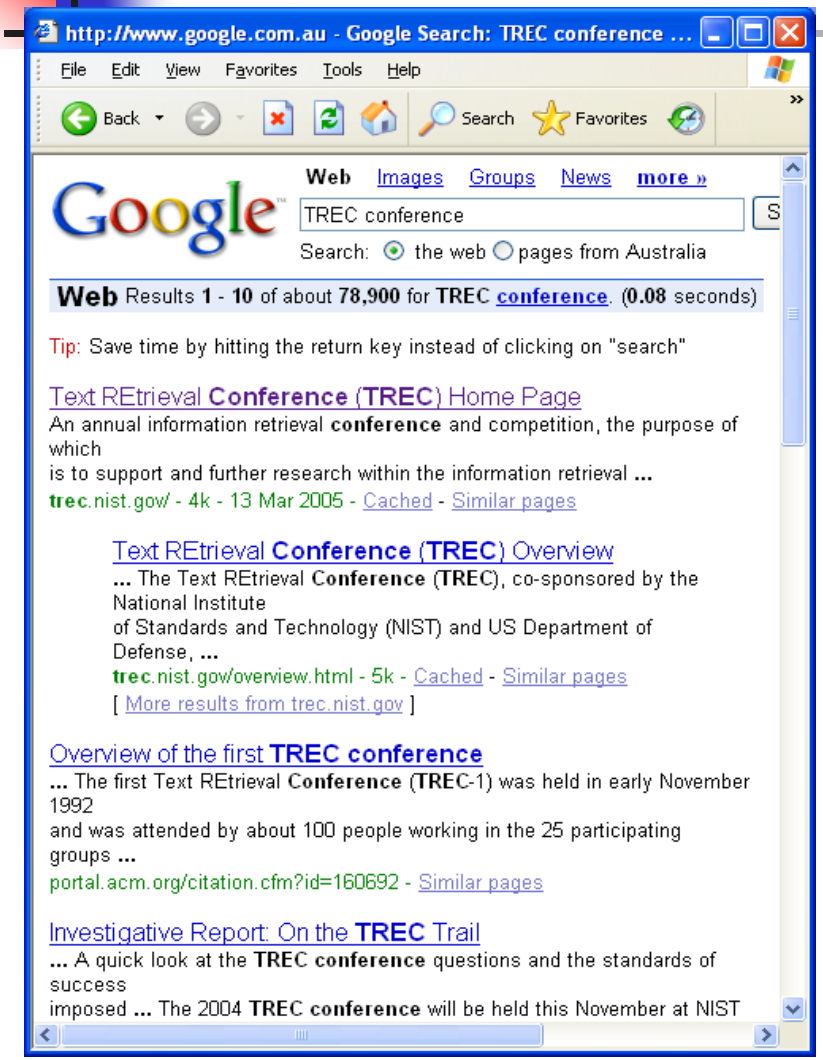
- Basic concepts
- Past present future
- Modeling
- Retrieval models

Information Retrieval

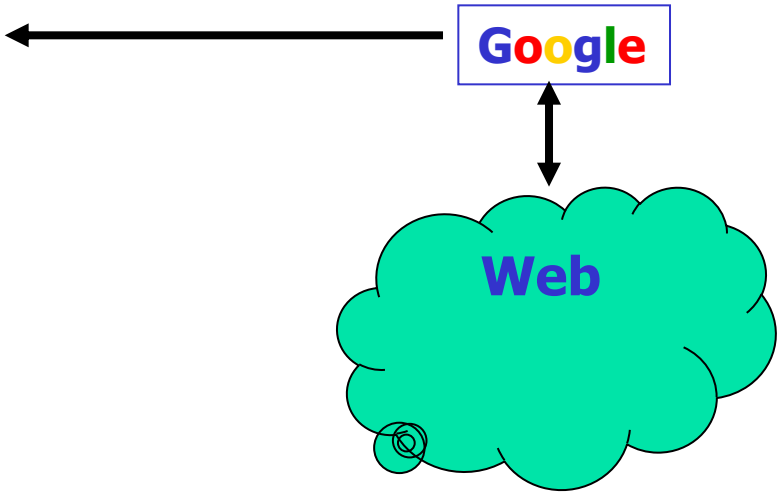
- **Goal** = find documents *relevant* to an information need from a large document set



Example


http://www.google.com.au - Google Search: TREC conference ...
File Edit View Favorites Tools Help
Back Search Favorites
Google
Web Images Groups News more »
TREC conference
Search: the web pages from Australia
Web Results 1 - 10 of about 78,900 for TREC conference. (0.08 seconds)
Tip: Save time by hitting the return key instead of clicking on "search"
Text REtrieval Conference (TREC) Home Page
An annual information retrieval conference and competition, the purpose of which
is to support and further research within the information retrieval ...
trec.nist.gov/ - 4k - 13 Mar 2005 - Cached - Similar pages
Text REtrieval Conference (TREC) Overview
... The Text REtrieval Conference (TREC), co-sponsored by the National Institute
of Standards and Technology (NIST) and US Department of Defense, ...
trec.nist.gov/overview.html - 5k - Cached - Similar pages
[More results from trec.nist.gov]
Overview of the first TREC conference
... The first Text REtrieval Conference (TREC-1) was held in early November 1992
and was attended by about 100 people working in the 25 participating groups ...
portal.acm.org/citation.cfm?id=160692 - Similar pages
Investigative Report: On the TREC Trail
... A quick look at the TREC conference questions and the standards of success
imposed ... The 2004 TREC conference will be held this November at NIST


http://www.google.com.au - Google - Microsoft...
File Edit View Favorites Tools Help
Back Search
Google Australia
Web Images Groups News more »
TREC conference
Google Search I'm Feeling Lucky
Search: the web pages from Australia


Google
Web

IR problem

- First applications: in libraries (1950s)

  ISBN: 0-201-12227-8

  Author: Salton, Gerard

  Title: Automatic text processing: the transformation, analysis, and retrieval of information by computer

  Editor: Addison-Wesley

  Date: 1989

  Content: <Text>

- external attributes and internal attribute (content)
- Search by external attributes = Search in DB
- IR: search by content

Information versus Data Retrieval

- The user of an IR system is concerned more with retrieving information about a subject than with retrieving data that satisfies a given query.
- A data retrieval system, such as a relational database, deals with data that has a well defined structure and semantics, while an IR system deals with natural language text which is not well structured.
- In a data retrieval system, on the contrary, a single erroneous object among a thousand retrieved objects means total failure.

Unstructured Data vs Structured Data [IR vs. Databases]

- Structured data tends to refer to information in "tables"

Employee	Manager	Salary
Smith	Jones	50000
Chang	Smith	60000
Ivy	Smith	50000

Typically allows numerical range and exact match (for text) queries, e.g.,
Salary < 60000 AND Manager = Smith.

- Unstructured data typically refers to **free text.**
- It allows **Adhoc and More sophisticated queries**.
- Keyword queries including operators like **and, or, not** etc.
- More sophisticated **"concept" queries** e.g., find all web pages dealing with football matches.
- Unstructured data means **semantically raw and easy for computer.**
- It may be **flat** or having **structure** like
  - Document structure (headings, paragraphs, lists. . .)
  - Explicit markup formatting (e.g. in HTML, XML. . .)

User Information Need:

Information need is represente through a natural language declaration which can be semantic ambiguous and vague in nature.

For instance, {Select documents on India's foreign policy and relations, 1947-1972}.

- IR deals with retrieving information about a subject.
- Queries are in Natural Language and may be not well structured which results into semantically ambiguous.
- IR system must interpret the contents of the documents and rank them according to the degree of relevance of the user need.

User Information Need

User information need in the context of the World Wide Web:

Find all the pages (documents) containing information on college tennis teams which:

(1) are maintained by an university in the USA and

(2) participate in the NCAA tennis tournament.

The page must include information on the national ranking of the team in the last three years and the email or phone number of the team coach.

Above user information need **cannot be used directly** to request information using the current interfaces of Web search engines.

Instead, the user must first **translate this information need into a query (set of keywords and set of operators)** which can be processed by the search engine (or IR system).

This translation yields a set of keywords (or index terms) which summarizes the description of the user information need.

Given the user query, the key goal of an IR system is to retrieve information which might be useful or relevant to the user.

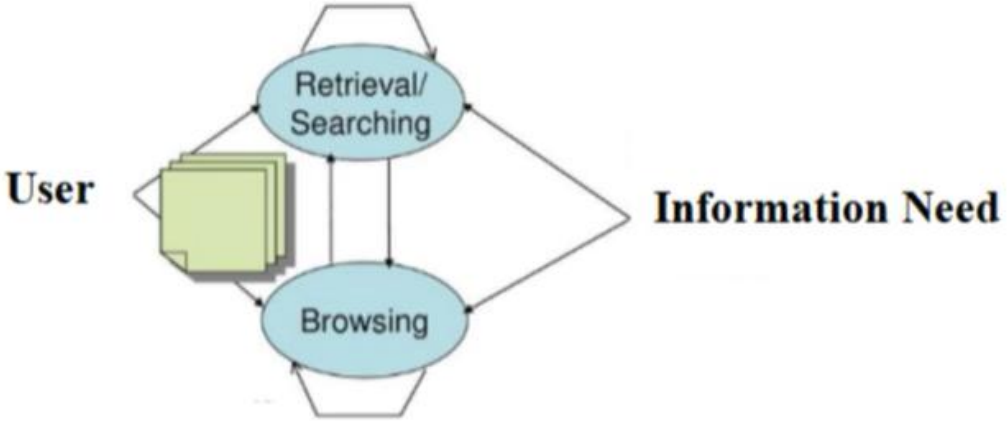
The emphasis is on the retrieval of information as opposed to the retrieval of data.

Information Retrieval Versus Data Retrieval

Information Retrieval	Data Retrieval
Information retrieval (IR) is a field of study dealing with the **representation, storage, organization of, and access to documents.**	Data retrieval deals with **obtaining data from a database management system such as RDBMS.**
Retrieves information about a **subject**.	User specifies query based on **column name (table) & keywords** and retrieves the data.
It is expressed in **NL**, therefore no strict syntax followed and small errors are likely to go unnoticed.	Wrong user query **(syntax error)** return failure.
User inormation need is specified in natural language therefore vague and semantically ambiguous.	Query has well-defined structure and semantics.
Does not provide a solution to the user of the database system.	Provides solutions to the user of the database system.

Information Retrieval	Data Retrieval
The results obtained are approximate matches.	The results obtained are exact matches.
Results are ranked by relevance.	Results are unordered by relevance unless specified by order clause
Probabilistic model	Deterministic model.
Natural language keywords are used	Query is in SQL language
Naive or expert users	Knowledgeable users or automatic processes
e.g. Various search engines, Google, Yahoo, Bing, Ask	e.g. Any database system like hospital, payroll etc
Bankruptcies in Mumbai branch for all types of banks	Find records with age > 50 and payment > 100K in Mumbai branch

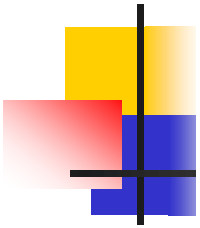
Basic Concepts of IR



The effective retrieval of relevant information is directly affected by

1) The user task

2) The logical view of the documents adopted by the retrieval system



1) The User Task

- First, the user information need is translated into a query by the user.
- The user uses natural language, boolean operators (and or not) to construct a queries for particular information need.
- Therefore, query may be semantically ambiguous,vague and adhoc in nature.
- Query quality may be different for naive users and expert users.
- Two Types: Browsing and Retrieval/Searching

Browsing:

- A user wants to search for something but ends up searching for another thing. This means that the user is browsing and not searching. (Search Topic is not fixed).
- The user can browse the documents when main objectives are not clear in the beginning and whose purpose might change during the interaction with the system.
- User information need related to cricket world cup: such as who won maximum time world cup, maximum runs and wickets in world cup, in which country final match was played
- From country, he/she may check tourism options of that country or any demographic, geographic, political status of that country.

Retrieval/Searching:

User gives a query to Information Systems. When the results are retrieved, user clicks on the relevant documents. Get more knowledge and enter refined queries with new keywords and continue searching on particular topic.

- Classic information retrieval systems normally allow information or data retrieval.
- Hypertext systems are usually tuned for providing quick browsing.
- Modern digital library and Web interfaces might attempt to combine these tasks (browsing and retrieval) to provide improved retrieval capabilities.
- However, combination of retrieval and browsing is not yet a well established approach

Pull Technology

User requests information in an interactive manner

Three types of retrieval tasks

- Browsing (hypertext)

- Retrieval (classical IR systems)

- Browsing and retrieval (modern digital libraries and web systems)

Push Technology

- Automatic and permanent pushing of information to user

- Software agents

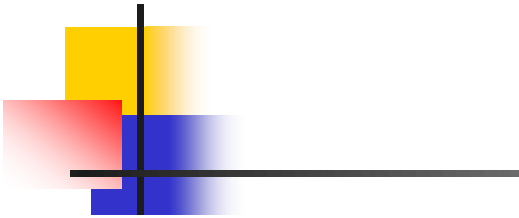
-example: news services, product information

- Filtering (retrieval task) relevant information for later inspection by user

2) The logical view of the documents adopted by the retrieval system

A) Index Terms or Keywords or Term Descriptor

- Keywords can be derived directly from the document's text (automatically by an algorithm) or given by a human subject/domain expert.
- These keywords provide a logical view of the document.
- **Advantages:** More concise representation, easy
- **Disadvantages:** Poor quality of retrieval



Term mismatch: It is possible that relevant documents containing synonymous terms will not be returned.

PRC vs. China

car vs. automobile

Ambiguity: It is possible to retrieve irrelevant documents that include ambiguous phrases.

Apple (company vs. fruit)

Java (programming language vs. Island)

B) Full Text:

- In this, all of the words that comprise the document's text are used as indexing keywords.
- For modern computers with high computation power, it is possible to represent a document by its full set of words.
- Most complete logical view of a document at the cost of high computation.

c) Intermediate View: Reduced set of keywords

However, with excessively massive document collections, even modern machines may required to decrease the list of representative keywords.

This can be accomplished by **removing**

1) Stopwords (such as articles and connectives)

2) Stemming (which reduces unique words to their common grammatical root)

3) Identifying noun groups (which removes adjectives, adverbs, and verbs).

4) Additionally, compression may be used.

These are known as text operations (or transformations).

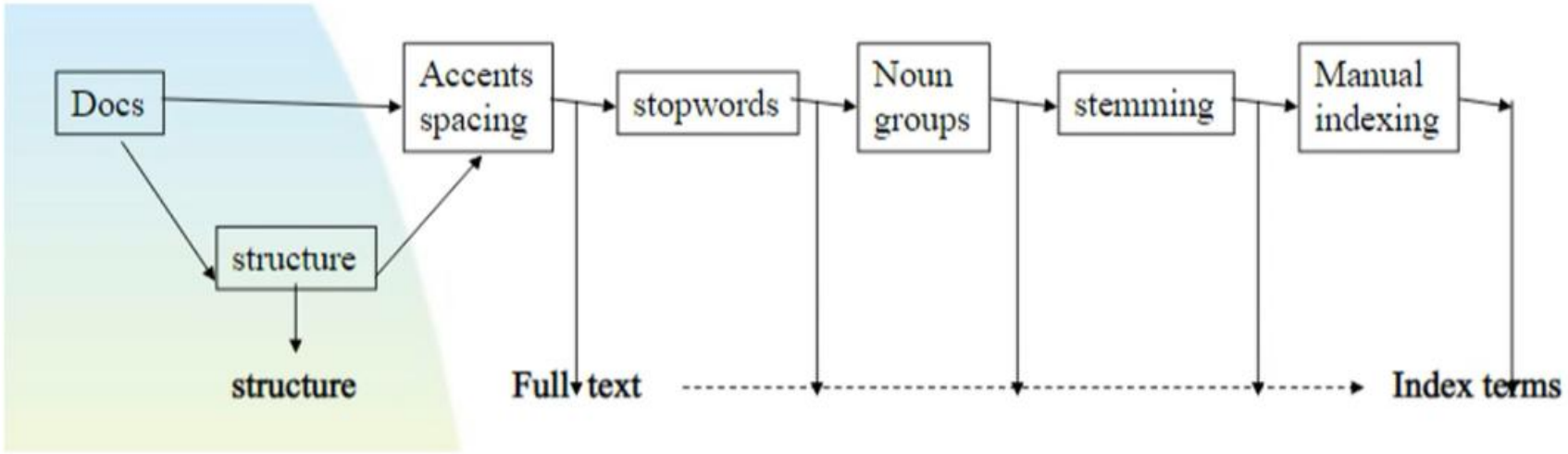
e.g. of Stopwords: **a, an, the, at,be,as etc**

e.g. of stemming: **play, played, playful, playing, plays: play**

e.g. of noun group: ***on the desk, bottle of milk, bowl of rice, kitchen table***

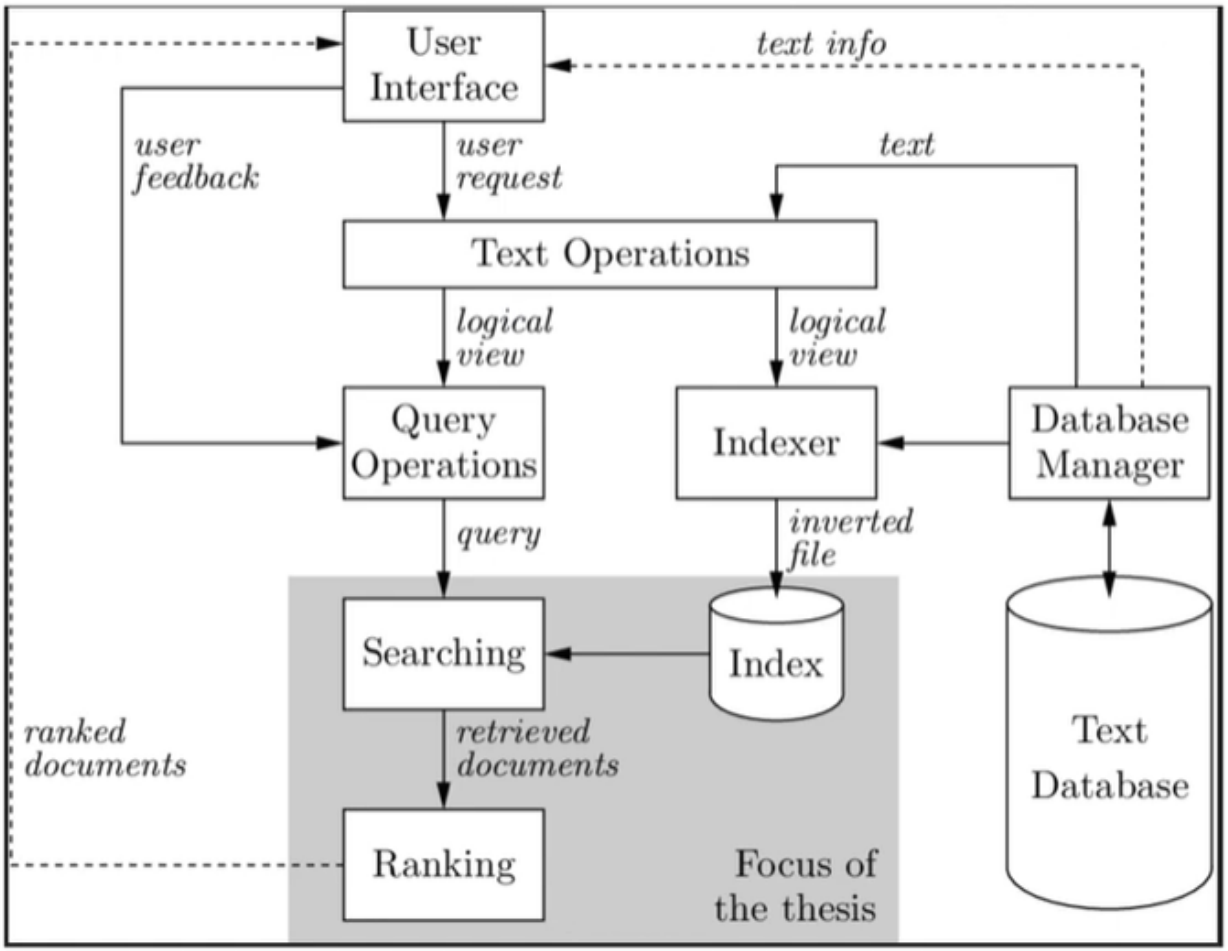
Compression: Compression on word as a symbol

Several intermediate logical views (of a document) might be adopted by an information retrieval system as shown in following figure.



- The retrieval system might also recognize the internal structure normally present in a document (e.g., chapters, sections, subsections, etc.).
- This information on the structure of the document might be quite useful and is required by **structured text retrieval models.**

The Retrieval Process



The process of retrieving information

1) Generating logical view of documents:

Before the retrieval process can even be initiated, it is necessary to define the **text database.** This is usuall

done by the **manager of the database**, which specifies the following:

(a) The documents to be used

(b) The operations to be performed on the text (stemming, stop word removal etc)

(c) The text model (i.e. the text structure and what elements can be retrieved)

The text operations transform the original documents and generate a logical view of them

2) Building an index of the text database:

- Once the logical view of the documents is defined, the database manager (using the DB Manager Module) builds an index of the text.
- An index is a critical data structure because it allows **fast searching over large volumes of data.**
- Different index structures might be used, but the most popular one is the ***inverted file.*** The resources **(time and storage space) spent on defining the text database** and building the index are amortized by querying the retrieval system many times

3) Initiating retrieval process by specifying a user need:

The user first specifies a user need by giving a query.

4)Generating logical view of user need (query):

Query is parsed and transformed by the same text operations applied to the text. Then, query operations might be applied before the actual query, which provides a system representation for the user need, is generated.

5) Retrieving documents by Processing the query:

The query is then processed to obtain the retrieved documents. Fast query processing is made possible by the index structure previously built.

6) Ranking documents upon relevance score:

Before been sent to the user, the retrieved documents are ranked according to a likelihood of relevance.

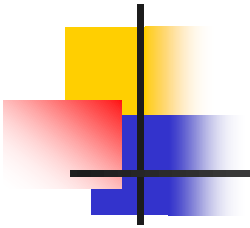
7) Initiating a user feedback cycle:

The user then examines the set of ranked documents in the search for useful information.

At this point, he/she might pinpoint a subset of the documents seen as definitely of interest and initiate a user feedback cycle.

In such a cycle, the system uses the documents selected by the user to change the query formulation.

Hopefully, this modified query is a better representation of the real user need.



Past, Present and Future

The index is an essential component and a key success factor for the IR system. In the beginning, indexes were manually generated by topic experts, much like the indexes created in libraries to categorise books.

Indexing can now be done automatically with the help of computers.

Two perspectives on the IR issue are presented in this aspect:

1) Computer Centered 2) Human Centered

1) Computer Centered :

IR problem consists mainly of **building up efficient indexes, processing user queries with high performance, and developing ranking algorithms which improve the `quality' of the answer set.**

2) Human Centered:

IR problem consists mainly of **studying the behavior of the user, of understanding his/her main needs, and of determining how such understanding affects the organization and operation of the retrieval system**.

According to this view, **keyword based query processing might be seen as a strategy which is unlikely to yield a good solution to the information retrieval problem** in the long run.

B) Information retrieval in the library:

Libraries were the first to adopt IR systems for information retrieval.

In first-generation, it consisted, automation of previous technologies, and the search was based on **author name and title**.

In the second generation, it included searching by **subject heading, keywords**, etc.

In the third generation, it consisted of **graphical interfaces, electronic forms, hypertext features, etc.**

e.g.Okapi (at City University, London), MELVYL (at University of California), and Cheshire II (at UC Berkeley).

C) Web and digital libraries:

Low cost, greater access, and publishing freedom have allowed people to use the Web (and modern digital libraries) as a highly interactive medium.

Such interactivity allows people to exchange messages, photos, documents, software, videos, and to `chat' in a convenient and low cost fashion.

In the future, **three main questions need** to be addressed.

First, in the dynamic world of the Web and of large digital libraries, which **techniques will allow retrieval of higher quality**?

Second, with the ever increasing demand for access, **quick response** is becoming more and more a pressing factor. Thus, **which techniques will yield faster indexes and smaller query response times?**

Third, the quality of the retrieval task is greatly affected by the user interaction with the system.

Thus, how will a better understanding of the **user behavior affect the design and deployment of new information retrieval strategies**?

Modeling Information Retrieval (IR)

- Taxonomy of Information Retrieval Models
- Retrieval/Search: Ad hoc and Filtering
- Formal characterization of IR models



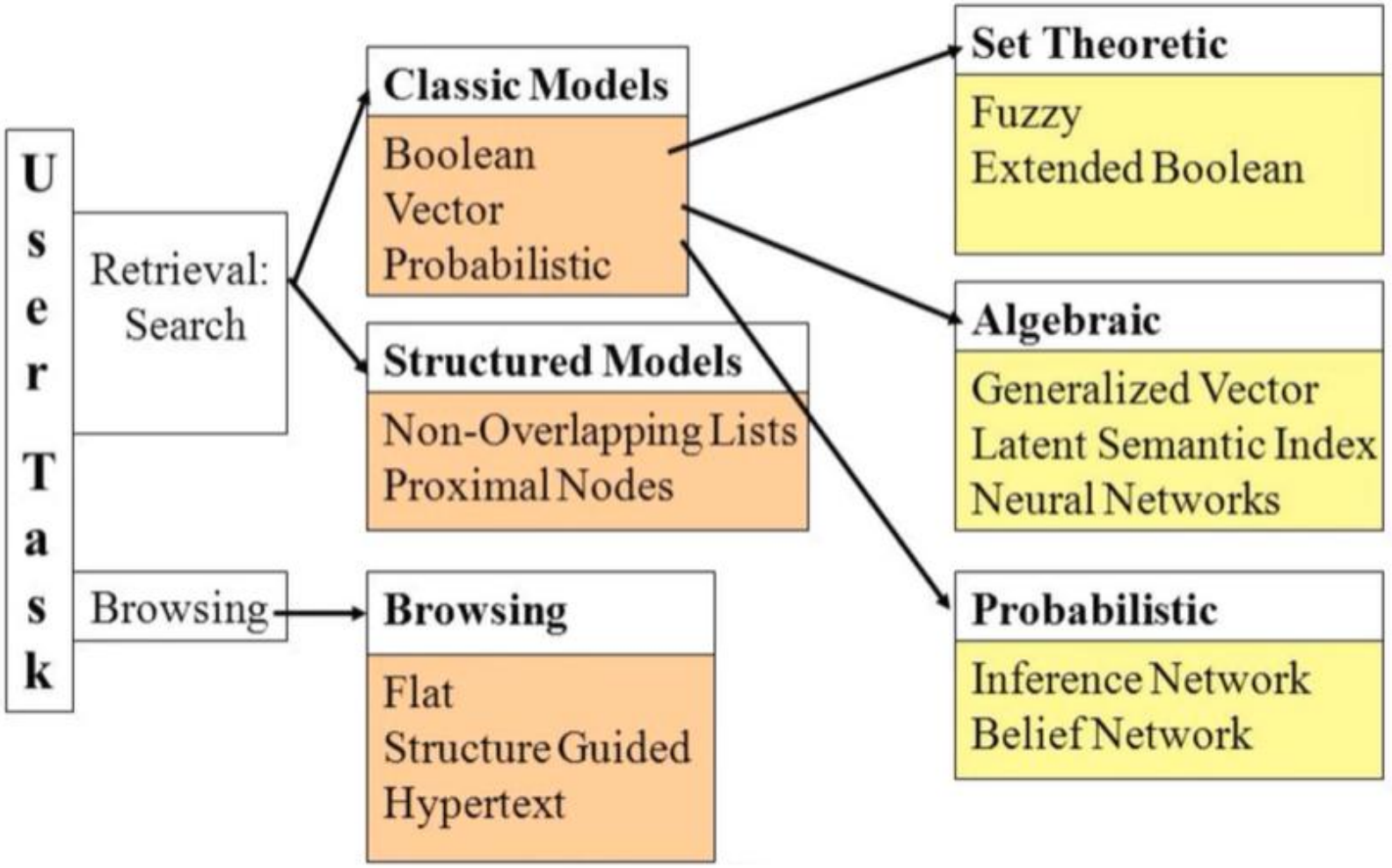
Modeling Information Retrieval (IR)

- The majority of the time, traditional information retrieval systems use **index words or keywords** for **indexing and retrieving documents**.

Advantages:

- Simple to extract (algorithm or manually)
- Sets of index phrases can naturally reflect the **semantic of the texts** and the **user's information needs**.

Taxonomy of Information Retrieval Models



Basic : Classic Model-Boolean, Extended: Set Theoretic

Documents and queries are represented as sets of index words in the Boolean model. As a result, the model is set theoretic.

Basic : Classic Model-Vector, Extended: Algebric

Documents and queries are represented as vectors in a t-dimensional space in the vector model. As a result, the model is algebraic.

Basic : Classic Model-Probablistic, Extended: Probablistic

The framework for modelling document and query representations in the probabilistic model is based on probability theory. As a result, the model is probabilistic.

	Index Terms	Full Text	Full Text+Structure
Retrieval/Searching	Classic Set Theoretic Algebraic Probabilistic	Classic Set Theoretic Algebraic Probabilistic	Structured
Browsing	Flat	Flat Hypertext	Structure Guided Hypertext

Retrieval models with distinct combinations of Document Logical View and User Task

Retrieval/Search : Ad hoc and Filtering

Ad hoc (Search): The documents in the collection remain relatively static while new queries are submitted to the system. **e.g. IR system of Libraries**

Routing (Filtering): The queries remain relatively static while new documents come into the system.

e.g. Search Engine

- User profile describes user's preference
- Keywords,
- Relevance feedback
- Dynamic keywords adjustment

In the filtering approach, "ranked retrieved documents" according to user information needs are displayed. Ranking is not shown to the user. The user will decide which documents are relevant or not.

In the routing approach, "ranked retrieved documents" according to user information needs are displayed with "rank". The main aim is that the user will choose the documents with a the higher rank

Formal characterization of IR models

Definition: An information retrieval model is a **quadruple (D,Q, F, R(q_i, d_j)]** where

(1) D is a set composed of logical views (or representations) for the documents in the collection.

(2) Q is a set composed of logical views (or representations) for the user in- formation needs(queries).

(3) F is a framework for modeling document representations, queries, and their relationships.

(4) R(q_i, d_j) is a ranking function which associates a real number with a query $q_i \in Q$ and a document representation $d_j \in D$. Such ranking defines an ordering among the documents with regard to the query q_i.

- Each document is described by a set of representative keywords called **index terms.**
- Assign a **numerical weights** to distinct relevance between index terms.
- Three classic models: **Boolean, vector, probabilistic**
- To decide importance of keyword is **difficult and tricky.**
- For that it is required to understand terms **semantics as well as its distribution in all documents.**
- Weights are used to quantify the importance of the index terms for describing the document contents.

Classic IR Model: Boolean

- It is the simpleest model
- Based on concepts of Set Theory and Boolean Logic (**and, or and not**)
- Popular till 1990's
- Used in WestLaw- the largest commercial legal search service

Query $Q = t_i \wedge t_j$ Here t_i and t_j are key terms

Documents with both t_i and t_j are retrieved.

$Q = t_i \vee t_j$ **Documents with either t_i or t_j are retrieved.**

$Q = \sim t_i$ **Documents without t_i are retrieved.**

Terms	Notation & Formula
q	User information need (query)
Document representation	d_j
Index Term weight for Document	i-keyword, t-total keywords, j -document $W_{i,j} \in \{0,1\}$
Query is expressed in DNF (Disjunctive Normal Form).	$\overrightarrow{Q_{dnf}}$
Each part of query is represented by	q_{cc}
Similarity Function is given by	$sim(d_j, q) = 1$ if $\exists\, q_{cc} \mid (q_{cc} \in \overrightarrow{Q_{dnf}}) \wedge (\forall k_i, g_i(\vec{d_j}) = g_i(q_{cc}))$

- If query is in **Disjunctive Normal Form (DNF)**, we can directly find out similarity between **document and query**.
- Because satisfiability of formulas in **DNF** can be achieved in **linear time**.
- If query is in **Conjunctive Normal Form (CNF)**, we have to convert first from **CNF to DNF**. Converting a formula to DNF (sum ofproduct terms) involves using logical equivalences, such as double negation elimination, De Morgan's laws, and the distributive law.

D1	Big cats are nice and funny
D2	Small dogs are better than big dog
D3	Small cats are afraid of small dogs
D4	Big cats are not afraid of small dogs
D5	Funny cats are not afraid of small dogs

Step 1: Stemming and Stop Word Removal

Step 2: Formation of Term × Document Matrix

Step 3: Check user query is in DNF form. If not convert to DNF

Step 4: Find similaity between query and each document by applying similairt function

After stemming and stop word removal	Big, cat, nice, funny, small, dog,better, afraid

Step 2: Formation of Term × Document Matrix

Keywrds\Documents	D1	D2	D3	D4	D5
big (K1)	1	1		1	
cat (K2)	1		1	1	1
nice (K3)	1				
funny (K4)	1				1
small (K5)		1	1	1	1
dog (K6)		1	1	1	1
better (K7)		1			
afraid (K8)			1	1	1

Step 3: Check user query is in DNF form. If not convert to DNF

$q = \text{Funny} \wedge \text{Dog}$ $\quad \overrightarrow{Q_{dnf}} = \text{Funny} \wedge \text{Dog}$ $\quad q_{cc} = \text{Funny} \wedge \text{Dog}$

Already in DNF Form

Step 4: Find similaity between query and each document by applying similairt function

$q_{cc} = \text{Funny} \wedge \text{Dog}$	K4	K6
D1	1	0
D2	0	1
D3	0	1
D4	0	1
D5	**1**	**1**

$$sim(d_j, q) = 1 \text{ if } \exists\, q_{cc} | (q_{cc} \in \overrightarrow{Q_{dnf}}) \wedge (\forall k_i, g_i(\vec{d_j}) = g_i(q_{cc}))$$

Two keywords

First is k4 and second is K6

For query, $\mathbf{g_i(q_{cc})= (1,1)}$ as they are present

Only for D5, $g_i(D5)$= **(1,1)** as they are present

Document D5 gets retrieved

Step 3: Check user query is in DNF form. If not convert to DNF

$q = \text{Funny} \vee \text{Dog}$ $\quad \overrightarrow{Q_{dnf}} = \text{Funny} \vee \text{Dog}$ $\quad q_{cc} = \text{Funny} \vee \text{Dog}$

Already in DNF Form

Step 4: Find similaity between query and each document by applying similairt function

$q_{cc} = \text{Funny} \vee \text{Dog}$	K4	K6
D1	1	0
D2	0	1
D3	0	1
D4	0	1
D5	1	1

$$sim(d_j, q) = 1 \text{ if } \exists\, q_{cc} | (q_{cc} \in \overrightarrow{Q_{dnf}}) \wedge (\forall k_i, g_i(\vec{d_j}) = g_i(q_{cc}))$$

Two keywords

First is k4 and second is K6

For query, $\mathbf{g_i(q_{cc}) = (1,1)}$ as they are present

For all documents, $\mathbf{g_i(Dj) = (1,1)}$, where j is from 1 to 5

Documents D1, D2, D3, D4 and D5 gets retrieved

Step 3: Check user query is in DNF form. If not convert to DNF

$$q = \text{Big} \wedge \text{Dog} \wedge \sim \text{Funny}$$

$$\overrightarrow{Q_{dnf}} = \text{Big} \wedge \text{Dog} \wedge \sim \text{Funny}$$

Already in DNF Form

$$q_{cc} = \text{Big} \wedge \text{Dog} \wedge \sim \text{Funny}$$

Step 4: Find similaity between query and each document by applying similairt function

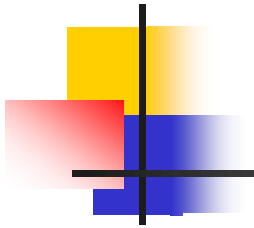
$q_{cc} = \text{Big} \wedge \text{Dog} \wedge \sim \text{Funny}$	K1	K6	~K4
D1	1	0	1
D2	**1**	**1**	**0**
D3	0	1	0
D4	**1**	**1**	**0**
D5	0	1	1

Three keywords: k1, k6 and ~k4

For query, **$g_i(q_{cc})$= 1 for (1,1,0)**

Only for D2 and D4 matches

Documents D2 and D4 gets retrieved



Step 3: Check user query is in DNF form. If not convert to DNF

$$q = \text{Cat} \wedge (\text{Nice} \vee \text{Afraid})$$

Not in DNF Form (CNF)

$$q = (\text{Cat} \wedge \text{Nice}) \vee (\text{Cat} \wedge \text{Afraid})$$

DNF Form

$$\overrightarrow{Q_{dnf}} = (\underbrace{\text{Cat} \wedge \text{Nice}}_{q_{cc}}) \vee (\underbrace{\text{Cat} \wedge \text{Afraid}}_{q_{cc}})$$

Step 4: Find similaity between query and each document by applying similairt function

$$\overrightarrow{Q_{dnf}} = (\text{Cat} \wedge \text{Nice}) \vee (\text{Cat} \wedge \text{Afraid})$$

q_{cc} q_{cc}

D1, D3, D4 and D5 retrieved

$q_{cc} = (\text{Cat} \wedge \text{Nice})$	K2	K3
D1	**1**	**1**
D2	0	0
D3	1	0
D4	1	0
D5	1	0

$q_{cc} = (\text{Cat} \wedge \text{Afraid})$	K2	K8
D1	1	0
D2	0	0
D3	**1**	**1**
D4	**1**	**1**
D5	**1**	**1**

Exercise 2

Document	Content
D1	Computer Information Retrieval
D2	Computer Retrieval
D3	Information
D4	Computer Information

Step 1: Stemming and Stop Word Removal

Step 2: Formation of Term × Document Matrix

Step 3: Check user query is in DNF form. If not convert to DNF

Step 4: Find similaity between query and each document by applying similairt function

Step 1: Stemming and Stop Word Removal

After stemming and stop word removal	Computer, Information, Retrieval

Step 2: Formation of Term × Document Matrix

Keywrds\Documents	D1	D2	D3	D4
Computer (K1)	1	1	0	1
Information (K2)	1	0	1	1
Retrieval (K3)	1	1	0	0

Step 3: Check user query is in DNF form. If not convert to DNF

$$q = \text{information} \wedge \text{retrieval}$$

Already in DNF Form

$\overrightarrow{Q_{dnf}} = \text{information} \wedge \text{retrieval}$ $q_{cc} = \text{information} \wedge \text{retrieval}$

Step 4: Find similaity between query and each document by applying similairt function

$q_{cc} = \text{information} \wedge \text{retrieval}$	K2	K3
D1	**1**	**1**
D2	0	1
D3	1	0
D4	1	0

Two keywords

First is k2 and second is K3

For query, $\mathbf{g_i(q_{cc})= (1,1)}$ as they are present

Only for D1, $\mathbf{g_i(D1)= (1,1)}$ as they are present

Documents D1 gets retrieved

$q = \text{information} \wedge (\text{computer} \vee \text{retrieval})$

$q = (\text{information} \wedge \text{computer}) \vee (\text{information} \wedge \text{retrieval})$

$$\overrightarrow{Q_{dnf}} = \underset{q_{cc}}{(\text{information} \wedge \text{computer})} \vee \underset{q_{cc}}{(\text{information} \wedge \text{retrieval})}$$

$q_{cc} =$ information $\wedge$ computer	K2	K1
D1	**1**	**1**
D2	0	1
D3	1	0
D4	**1**	**1**

$q_{cc} =$ information $\wedge$ retrieval	K2	k3
D1	**1**	**1**
D2	0	1
D3	1	0
D4	1	0

Documents D1, D4 gets retrieved

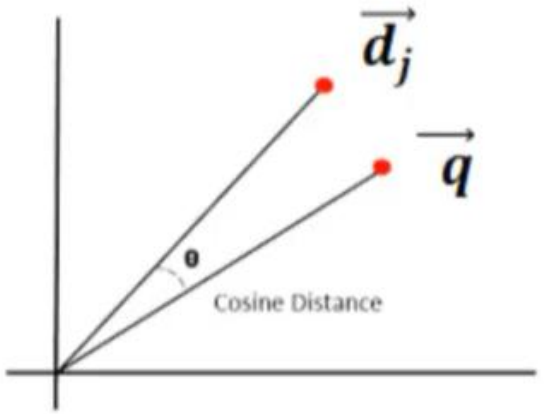
Vector model

Terms	Notation & Formula
Intra-Cluster Similarity	$f_{i,j} = \frac{freq_{i,j}}{maxl\ freq_{i,j}}$ $freq_{i,j}$: Term frequency of k_i in j doc. maxl $freq_{i,j}$:Term frequency of k_i in j doc. $f_{i,j}$:Normalized frequency
Inter-Cluster Similarity	$idf_i = log\frac{N}{n_i}$ or $idf_i = log(1 + \frac{N}{n_i})$ N- the total number of documents in the system n_i the number of documents in which the index term k_i appears.
$w_{i,j}$	$w_{i,j} = f_{i,j} \times idf_i$
$w_{i,q}$	$w_{i,q} = (0.5 + f_{i,q}) \times idf_i$ $f_{i,q} = \frac{freq_{i,q}}{maxl\ freq_{i,q}}$

Degree of similarity between $\vec{q}$ and $\vec{d_j}$

$$sim(d_j, q) = \frac{\vec{d_j} \cdot \vec{q}}{|\vec{d_j}| \times |\vec{q}|}$$

$$sim(d_j, q) = \frac{\sum_{i=1}^{t} w_{i,j} \times w_{i,q}}{\sqrt{\sum_{i=1}^{t} w_{i,j}^2} \times \sqrt{\sum_{i=1}^{t} w_{i,q}^2}}$$



The factor $\vec{q}$ does not affect the ranking because it is the same for all documents.

The factor $\vec{d_j}$ provides a normalization in the space of the documents.

Exercise 1

Document	Content
Doc1	Cat runs behind rat
Doc2	Dog runs behind cat
Doc3	Dog , cat and rat run behind cheese
Query	Query: cat cheese
After stemming & stop word removal	cat, run, behind, rat,dog, cheese

Step 1: Stemming and Stop Word Removal

Step 2: Intra Cluster Similarity

Step 3: Inter cluster Similarity

After stemming & stop word removal	cat, run, behind, rat,dog, cheese

Sr. No.	Index Term or Keywords	Doc1	Doc2	Doc3	Query: cat, cheese
1	cat	1	1	1	1
2	run	1	1	1	0
3	behind	1	1	1	0
4	rat	1	0	1	0
5	dog	0	1	1	0
6	cheese	0	0	1	1
	Max Frequency	1	1	1	1

$$f_{i,j} = \frac{freq_{i,j}}{maxl\ freq_{i,j}}$$

$$f_{i,q} = \frac{freq_{i,q}}{maxl\ freq_{i,q}}$$

Inter cluster similarity

Sr. No.	Index Term or Keywords	Doc1	Doc2	Doc3	Count	$q_{i,j}$	$idf_i = log(1+\frac{N}{n_i})$
1	cat	1	1	1	3	1.5	0.30
2	run	1	1	1	3	0.5	0.30
3	behind	1	1	1	3	0.5	0.30
4	rat	1	0	1	2	0.5	0.40
5	dog	0	1	1	2	0.5	0.40
6	cheese	0	0	1	1	1.5	0.60

$$idf_i = log(1+\frac{N}{n_i})$$

$$(0.5 + f_{i,q})$$

Sr. No.	Index Term or Keywords	Doc1	Doc2	Doc3	Query: cat, cheese
1	cat	0.30	0.30	0.30	0.45
2	run	0.30	0.30	0.30	0.15
3	behind	0.30	0.30	0.30	0.15
4	rat	0.40	0.00	0.40	0.20
5	dog	0.00	0.40	0.40	0.20
6	cheese	0.00	0.00	0.60	0.90

$$w_{i,j} = f_{i,j} \times idf_i$$

$$w_{i,q} = (0.5 + f_{i,q}) \times idf_i$$

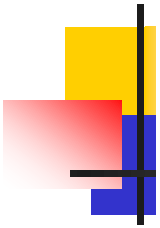
$$sim(d_j, q) = \frac{\vec{d_j} \cdot \vec{q}}{|\vec{d_j}| \times |\vec{q}|}$$

$$sim(d_j, q) = \frac{\sum_{i=1}^{t} w_{i,j} \times w_{i,q}}{\sqrt{\sum_{i=1}^{t} w_{i,j}^2} \times \sqrt{\sum_{i=1}^{t} w_{i,q}^2}}$$

Sr. No.	Index Term or Keywords	Doc1	Query: cat, cheese	
1	cat	0.30	0.45	0.14
2	run	0.30	0.15	0.05
3	behind	0.30	0.15	0.05
4	rat	0.40	0.20	0.08
5	dog	0.00	0.20	0.00
6	cheese	0.00	0.90	0.00
		0.66	1.07	0.31

$\sqrt{\sum_{i=1}^{t} w_{i,j}^2}$ $\sqrt{\sum_{i=1}^{t} w_{i,q}^2}$ $\sum_{i=1}^{t} w_{i,j} \times w_{i,q}$

$$sim(Doc1, q) = \frac{0.31}{0.66 \times 1.07} = 0.44$$



$$sim(d_j, q) = \frac{\vec{d_j} \cdot \vec{q}}{|\vec{d_j}| \times |\vec{q}|}$$

$$sim(d_j, q) = \frac{\sum_{i=1}^{t} w_{i,j} \times w_{i,q}}{\sqrt{\sum_{i=1}^{t} w_{i,j}^2} \times \sqrt{\sum_{i=1}^{t} w_{i,q}^2}}$$

Sr. No.	Index Term or Keywords	Doc2	Query: cat, cheese	
1	cat	0.30	0.45	0.14
2	run	0.30	0.15	0.05
3	behind	0.30	0.15	0.05
4	rat	0.00	0.20	0.00
5	dog	0.40	0.20	0.08
6	cheese	0.00	0.90	0.00
		0.66	1.07	0.31

$\sqrt{\sum_{i=1}^{t} w_{i,j}^2}$ $\sqrt{\sum_{i=1}^{t} w_{i,q}^2}$ $\sum_{i=1}^{t} w_{i,j} \times w_{i,q}$

$$sim(Doc2, q) = \frac{0.31}{0.66 \times 1.07} = 0.44$$

$$sim(d_j, q) = \frac{\vec{d_j} \cdot \vec{q}}{|\vec{d_j}| \times |\vec{q}|}$$

$$sim(d_j, q) = \frac{\sum_{i=1}^{t} w_{i,j} \times w_{i,q}}{\sqrt{\sum_{i=1}^{t} w_{i,j}^2} \times \sqrt{\sum_{i=1}^{t} w_{i,q}^2}}$$

Sr. No.	Index Term or Keywords	Doc3	Query: cat, cheese	
1	cat	0.30	0.45	0.14
2	run	0.30	0.15	0.05
3	behind	0.30	0.15	0.05
4	rat	0.40	0.20	0.08
5	dog	0.40	0.20	0.08
6	cheese	0.60	0.90	0.54
		0.98	1.07	0.93

$\sqrt{\sum_{i=1}^{t} w_{i,j}^2}$ $\sqrt{\sum_{i=1}^{t} w_{i,q}^2}$ $\sum_{i=1}^{t} w_{i,j} \times w_{i,q}$

$$sim(Doc3, q) = \frac{0.93}{0.98 \times 1.07} = 0.89$$

Document	Similarity Score
Doc1	**0.44**
Doc2	**0.44**
Doc3	**0.89**

Advantages:

- The retrieval performance is improved by its term-weighting method.
- With the help of its partial matching approach, documents that roughly match the query conditions can be retrieved.
- The documents are sorted using its cosine ranking formula based on how similar they are to the query.

Disadvantage

- The belief that index terms are independent of one another

Exercise 1

Document	Content
Doc1	Binary Inormation Retrieval Model
Doc2	Vector Retrieval Model
Doc3	Extended Binary Model
Query	Query: Extended Binary

Step 1: Stemming and Stop Word Removal

Step 2: Intra Cluster Similarity

Step 3: Inter cluster Similarity

Step 4: Cosine similarity between document vector and query vector

Sr. No.	Index Term or Keywords	Doc1	Doc2	Doc3	Query: Extended Binary
1	Binary	1	0	1	1
2	Information	1	0	0	0
3	Retrieval	1	1	0	0
4	Model	1	1	1	0
5	Vector	0	1	0	0
6	Extended	0	0	1	1
	Max Frequency	1	1	1	1

$$f_{i,j} = \frac{freq_{i,j}}{maxl\ freq_{i,j}}$$

$$f_{i,q} = \frac{freq_{i,q}}{maxl\ freq_{i,q}}$$

Sr. No.	Index Term or Keywords	Doc1	Doc2	Doc3	Query: Extended Binary	Count	log(1+3/count)
1	Binary	1	0	1	1.5	2	0.40
2	Information	1	0	0	0.5	1	0.60
3	Retrieval	1	1	0	0.5	2	0.40
4	Model	1	1	1	0.5	3	0.30
5	Vector	0	1	0	0.5	1	0.60
6	Extended	0	0	1	1.5	1	0.60

$$idf_i = log(1 + \frac{N}{n_i})$$

$$(0.5 + f_{i,q})$$

Sr. No.	Index Term or Keywords	Doc1	Doc2	Doc3	Query: Extended Binary	Count	log(1+3/count)
1	Binary	0.40	0.00	0.40	0.60	2	0.40
2	Information	0.60	0.00	0.00	0.30	1	0.60
3	Retrieval	0.40	0.40	0.00	0.20	2	0.40
4	Model	0.30	0.30	0.30	0.15	3	0.30
5	Vector	0.00	0.60	0.00	0.30	1	0.60
6	Extended	0.00	0.00	0.60	0.90	1	0.60

$$w_{i,j} = f_{i,j} \times idf_i$$

$$w_{i,q} = (0.5 + f_{i,q}) \times idf_i$$

Sr. No.	Index Term or Keywords	Doc1	Doc2	Doc3	Query: Extended Binary	Count	log(1+3/count)
1	Binary	0.40	0.00	0.40	0.60	2	0.40
2	Information	0.60	0.00	0.00	0.30	1	0.60
3	Retrieval	0.40	0.40	0.00	0.20	2	0.40
4	Model	0.30	0.30	0.30	0.15	3	0.30
5	Vector	0.00	0.60	0.00	0.30	1	0.60
6	Extended	0.00	0.00	0.60	0.90	1	0.60

$$w_{i,j} = f_{i,j} \times idf_i$$

$$w_{i,q} = (0.5 + f_{i,q}) \times idf_i$$

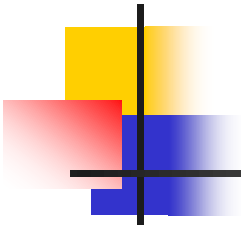
$$sim(d_j, q) = \frac{\vec{d_j} \cdot \vec{q}}{|\vec{d_j}| \times |\vec{q}|}$$

$$sim(d_j, q) = \frac{\sum_{i=1}^{t} w_{i,j} \times w_{i,q}}{\sqrt{\sum_{i=1}^{t} w_{i,j}^2} \times \sqrt{\sum_{i=1}^{t} w_{i,q}^2}}$$

Sr. No.	Index Term or Keywords	Doc1	Query	
1	Binary	0.40	0.60	0.24
2	Information	0.60	0.30	0.18
3	Retrieval	0.40	0.20	0.08
4	Model	0.30	0.15	0.05
5	Vector	0.00	0.30	0.00
6	Extended	0.00	0.90	0.00
		0.88	**1.19**	**0.54**

$\sqrt{\sum_{i=1}^{t} w_{i,j}^2}$ $\sqrt{\sum_{i=1}^{t} w_{i,q}^2}$ $\sum_{i=1}^{t} w_{i,j} \times w_{i,q}$

$$sim(Doc1, q) = \frac{0.54}{0.88 \times 1.19} = 0.52$$



Document	Content
Doc1	Binary Inormation Retrieval Model
Doc2	Vector Retrieval Model
Doc3	Extended Binary Model
Query	Query: Extended Binary

Document	Similarity Score
Doc1	0.52
Doc2	0.33
Doc3	0.89

Introduction to Probabilistic Model

- Every document is characterized by keywords.
- Generally for a given user query, there is a set of documents that contains keywords similar to user query keywords i.e. relevant documents and no other (irrelvant documents). Let us refer to this set of documents as the Ideal Answer Set.
- Given the description(keywords) of this ideal answer set, we would have no problems in retrieving its documents. Thus, we can think of the **querying process as a process of specifying the properties of an ideal answer set.**
- In reality, repository contains unlimited documents, it is not possible to give its properties (keywords)
- For e.g. 10 lacs docs are there, total 1k keywords are there. It is not possible to for the enduser to remember these keywords(or their synonyms). The problem is that we do not know exactly what these properties are.

- All we know is that there are **index terms (given by user query) whose semantics should be used to characterize these properties.**
- Since these properties are not known at query time, an effort has to be made at initially guess what they could be.
- This initial guess allows us to generate a preliminary probabilistic description of the ideal answer set which is used to retrieve the first set of documents.
- Interaction with the user is then initiated with the purpose of improving the probabilistic description of the ideal answer set. Such interaction could proceed as follows.
- The user takes a look at the retrieved documents and decides which ones are relevant and which ones are not (in truth, only the first top documents need to be examined). The system then uses this information to refine the description of the ideal answer set.
- By repeating this process many times, it is expected that such a description will evolve and become closer to the real description of the ideal answer set. Thus, one should always have in mind the need to guess at the beginning the description of the ideal answer set.

Probabilistic Ranking Principle

- Based on concept of conditional probability and Log of Odds
- Tries to estimate the probability that a document will be relevant to a user query
- Assumes that this probability depends on the query and document representations only
- The ideal answer set, referred to as R, should maximize the probability of relevance

Bayes' Rule of Conditional Probability and Log Odds Ratio

Let a and b are two events

$$p(a \mid b)p(b) = p(a \cap b) = p(b \mid a)p(a)$$

$$p(a \mid b) = \frac{p(b \mid a)p(a)}{p(b)}$$

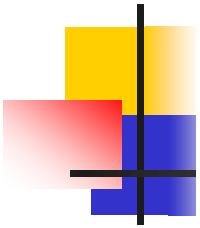
$$p(\bar{a} \mid b) = \frac{p(b \mid \bar{a})p(\bar{a})}{p(b)}$$

$$\log(\text{odds}) = \text{logit}(x) = \log\left(\frac{x}{(1-x)}\right)$$

$$\text{Log Odds Ratio} = \text{logit}\left(\frac{p_x}{p_y}\right) = \log\left(\frac{\frac{p_x}{(1-p_x)}}{\frac{p_y}{(1-p_y)}}\right)$$

$$\text{Log Odds Ratio} = \log\left(\frac{p_x}{(1-p_x)} \times \frac{(1-p_y)}{p_y}\right)$$

$$\text{Log Odds Ratio} = \log\left(\frac{p_x}{(1-p_x)}\right) + log\left(\frac{(1-p_y)}{p_y}\right)$$



Documents: D1, D2, D3, D4, D5

Query (K1, K2)

Relevant (R)=D2, D4

Irrelevant ($\bar{R}$)=D1, D3, D5

Doc	K1	K2	R/$\bar{R}$	
D1	1	0	$\bar{R}$	
D2	0	1	R	Click
D3	1	0	$\bar{R}$	
D4	1	1	R	Click
D5	0	1	$\bar{R}$	

Doc	K1	K2	R/$\overline{R}$
D1	1	0	$\overline{R}$
D2	0	1	R
D3	1	0	$\overline{R}$
D4	1	1	R
D5	0	1	$\overline{R}$

Relevant	Irrelevant
$P(R) = 2/5$	$P(\overline{R}) = 3/5$
$P(K1\|R) = 1/2$	$P(K1\|\overline{R}) = 2/3$
$P(K2\|R) = 2/2$	$P(K2\|\overline{R}) = 1/3$

Prior Probability

Likelihood

Definition:

For the probabilistic model, the index term weight variables are all binary i.e.,

$w_{i,j} \in \{0, 1\}, w_{i,q} \in \{0, 1\}$.

A query **q** is a subset of index terms.

Let **R** be the set of documents known (or initially guessed) to be relevant.

Let $\bar{R}$ be the complement of R (i.e., the set of non-relevant documents).

Let $P(R|d_j)$ be the probability that the document d_j is relevant to the query **q** and $P(\bar{R}|d_j)$ be the probability that d_j is non-relevant to **q**.

The similarity **sim(d_j,q)** of the document d_j, to the query **q** is defined as the ratio

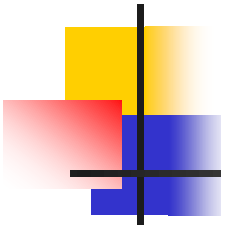
$$sim(\vec{d_j}, q) = \frac{Pr(R|\vec{d_j})}{Pr(\bar{R}|\vec{d_j})}$$

$$sim(\vec{d_j}, q) = \frac{Pr(\vec{d_j}|R) \times Pr(R)}{Pr(\vec{d_j}|\bar{R}) \times Pr(\bar{R})}$$

P(R) stands for the probability that a document randomly selected from the entire collection is relevant

P(d_j|R) stands for the probability of randomly selecting the document d_j, from the set R of relevant documents

12



$$\text{sim}(\vec{d_j}, q) = \frac{\Pr(\vec{d_j}|R) \times \Pr(R)}{\Pr(\vec{d_j}|\bar{R}) \times \Pr(\bar{R})}$$

$$\frac{A \times B}{C \times D} = \frac{log(AB)}{log(CD)} = \frac{log(A) + log(B)}{log(C) + log(D)}$$

$$\frac{A \times B}{C \times D} = log\, A + logB - logC - logD$$

$$\frac{A \times B}{C \times D} = log\, \frac{A}{C} + log\, \frac{B}{D}$$

$$\text{sim}(\vec{d_j}, q) \approx log\, \frac{\Pr(\vec{d_j}|R)}{\Pr(\vec{d_j}|\bar{R})} + log \frac{\Pr(R)}{\Pr(\bar{R})}$$

$$\text{sim}(\vec{d}_j, q) \approx log \frac{\Pr(\vec{d}_j|R)}{\Pr(\vec{d}_j|\bar{R})} + log \frac{\Pr(R)}{\Pr(\bar{R})}$$

Assuming independence of index terms and given $q=(d_1,d_2,d_3,....d_t)$

$$\Pr(\vec{d}_j|R) = \prod_{i=1}^{t} Pr(k_i = d_i|R) \qquad \Pr(\vec{d}_j|\bar{R}) = \prod_{i=1}^{t} Pr(k_i = d_i|\bar{R})$$

$$\text{sim}(\vec{d}_j, q) \approx log \frac{\prod_{i=1}^{t} Pr(k_i = d_i|R)}{\prod_{i=1}^{t} Pr(k_i = d_i|\bar{R})} + constants$$

$Pr(k_i|R)$ stands for the probability that the index term k_i, is present in a document randomly selected from the set R

$Pr(\bar{k}_i|R)$ stands for the probability that the index term k_i is not present in a document randomly selected from the set R

Odds of logs: Odds are defined as the probability that the event will occur divided by the probability that the event will not occur. Log-Odds ratio is simply the natural log of the odds ratio. An odds ratio is a relationship between the exposure of one variable and the occurrence of the other

$$\text{Log Odds Ratio} = \log\left(\frac{p_x}{(1-p_x)} \times \frac{(1-p_y)}{p_y}\right) \quad \text{Log Odds Ratio} = \log\left(\frac{p_x}{(1-p_x)}\right) + log\left(\frac{(1-p_y)}{p_y}\right)$$

$$\text{sim}(\vec{d_j}, q) \approx log \frac{\prod_{i=1}^{t} Pr(k_i = d_i|R)}{\prod_{i=1}^{t} Pr(k_i = d_i|\overline{R})} + constants$$

$$\because \Pr(k_i \mid R) + \Pr(\overline{k_i} \mid R) = 1$$

$$sim(\vec{d}_j, q) \approx \sum_{i=1}^{t} \left(\log \frac{P(k_i \mid R)}{1 - P(k_i \mid R)} + \log \frac{1 - P(k_i \mid \overline{R})}{P(k_i \mid \overline{R})} \right)$$

$$sim(d_j, q) \approx \sum_{i=1}^{t} w_{i,q} \times w_{i,j} \times log \frac{p(k_i|R)}{1 - p(k_i|R)} + log \frac{1 - p(k_i|\overline{R})}{p(k_i|\overline{R})}$$

Query (K1, K2)

Doc	K1	K2	R/$\bar{R}$
D1	1	0	$\bar{R}$
D2	0	1	R
D3	1	0	$\bar{R}$
D4	1	1	R
D5	0	1	$\bar{R}$

Relevant	Irrelevant
$P(R) = 2/5$	$P(\bar{R}) = 3/5$
$P(K1|R) = 1/2$	$P(K1|\bar{R}) = 2/3$
$P(K2|R) = 2/2$	$P(K2|\bar{R}) = 1/3$

Term	Doc1(1,0)	Query(1,1)	$: log \frac{p(k_i|R)}{1-p(k_i|R)}$	$log \frac{1-p(k_i|\bar{R})}{p(k_i|\bar{R})}$	Summation logs of odd	Final Answer
K1	1	1	0	-0.30	-0.30	-0.3
K2	0	1	Dont Do	Dont Do	0	

Relevant	Irrelevant
$P(R) = 2/5$	$P(\bar{R}) = 3/5$
$P(K1|R) = 1/2$	$P(K1|\bar{R}) = 2/3$
$P(K2|R) = 2/2$	$P(K2|\bar{R}) = 1/3$

Term	Doc4(1,1)	Query(1,1)	$\log \frac{p(k_i|R)}{1-p(k_i|R)}$	$\log \frac{1-p(k_i|\bar{R})}{p(k_i|\bar{R})}$		
K1	1	1	0	-0.30	-0.30	0
K2	1	1	0	0..30	0.3	
Term	Doc5(0,1)	Query(1,1)	$\log \frac{p(k_i|R)}{1-p(k_i|R)}$	$\log \frac{1-p(k_i|\bar{R})}{p(k_i|\bar{R})}$		
K1	0	1	Dont Do	Dont Do	0.00	0.3
K2	1	1	0	0..30	0.3	



Document	Similarity Score
D1	-0.3
D2	0.3
D3	-0.3
D4	0
D5	0.3

Ranking D2, D5, D4,D1 and D3

In the very beginning (i.e., immediately after the query specification), there are no retrieved documents.

Therefore assumptions are as below

$$Pr(\vec{k_i}|R) = 0.5$$

$$Pr(\vec{k_i}|\bar{R}) = n_i/N$$

Same as idf (Inverse Document Frequency)

If n_i is present in all documents (N).

It indicates its common term.

Therefore its considered as Non relevant document.

N: Total number of documents

$\mathbf{n_i}$: The number of documents that contain $\mathbf{k_i}$

This probabilities are used before user mark relevant or irrelvant.

22

Improves Initial Ranking

- **V**: Initially retrieved set of Documents

V_i: Subset of documents retrieved that contain **k_i**.

After retrieval of initial documents, probabilities are reassigned as below

$$Pr(\vec{k_i}|R) = \frac{V_i}{V} \qquad Pr(\vec{k_i}|\bar{R}) = \frac{n_i - V_i}{N - V}$$

V:5 N:20

$$Pr(\vec{k_1}|R) = \frac{3}{5}$$

$$Pr(\vec{k_2}|R) = \frac{3}{5}$$

$$Pr(\vec{k_1}|\bar{R}) = \frac{10-3}{20-5} = \frac{7}{15}$$

$$Pr(\vec{k_1}|\bar{R}) = \frac{13-3}{20-5} = \frac{10}{15}$$

Doc	K1	K2	R/$\bar{R}$
D1	1	0	$\bar{R}$
D2	0	1	R
D3	1	0	$\bar{R}$
D4	1	1	R
D5	0	1	$\bar{R}$

23

Smoothing Operation

To avoid the problem with V=1 and $\mathbf{V_i=0}$ of

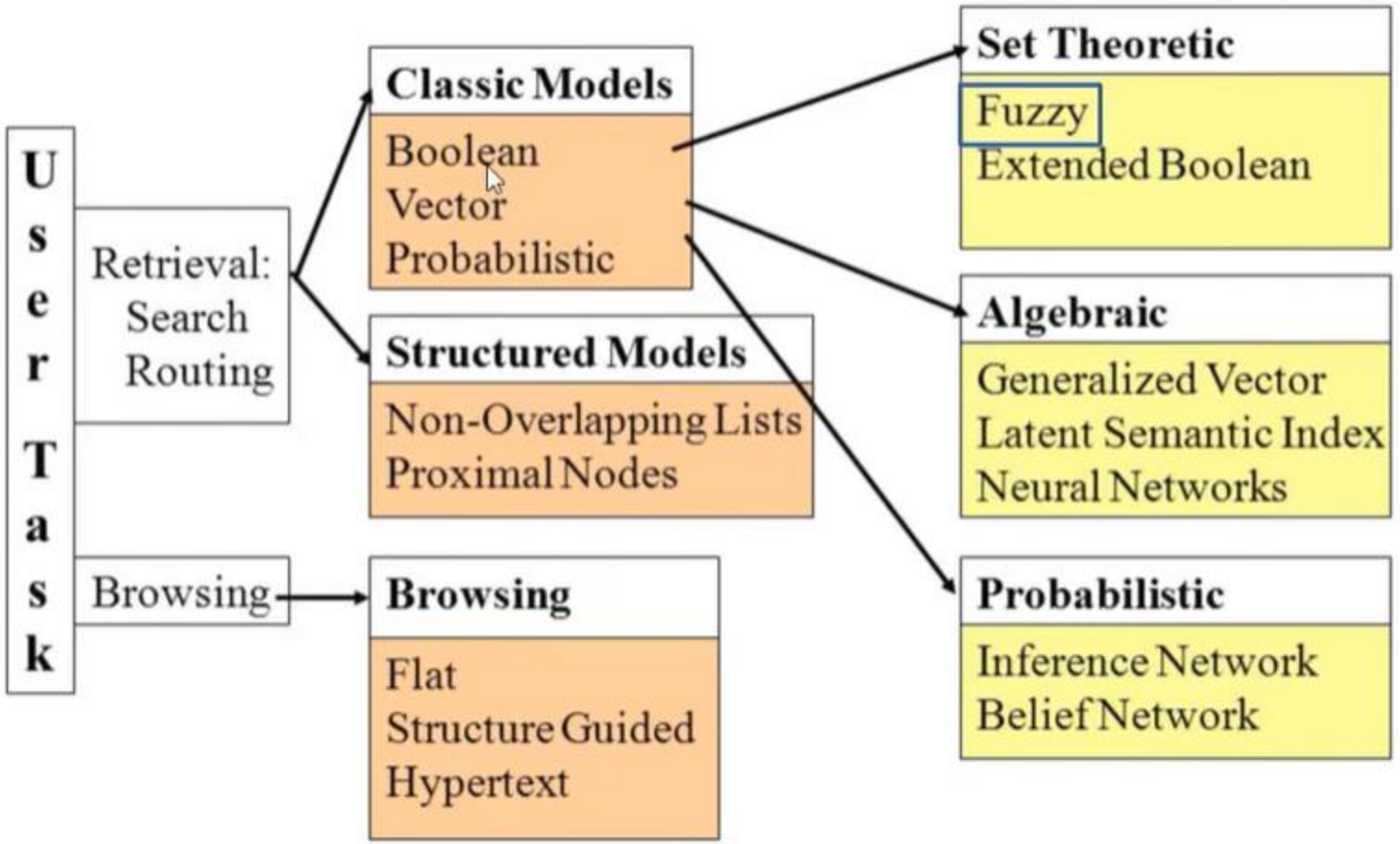
$$Pr(\overrightarrow{k_i}|R) = \frac{V_i+0.5}{V+1} \text{ or } \frac{V_i+n_i/N}{V+1}$$

$$Pr(\overrightarrow{k_1}|\bar{R}) = \frac{n_i - V_i + 0.5}{N - V + 1} \text{ or } \frac{n_i - V_i + n_i/N}{N - V + 1}$$

Imp. Note: Problem in this video is solved with conditional probabilities.

But if we implement algorithm, we can assign initial probabilities. It is not possible to solve manually.

Taxonomy of Information Retrieval Models



Fuzzy Model

- Framework for representing classes with ill-defined boundaries.
- The main aim is to introduce the concept of a degree of membership linked with set elements.
- This degree of membership ranges from 0 to 1 and can be used to model the concept of marginal membership.
- Thus, membership is vague (not clear) concept, as opposed to the crisp concept imposed by basic Boolean logic.

Classical Set/ Crisp Set Vs Fuzzy Set

Classical Set/ Crisp Set

Universal Set/Universe of Discourse (X): A set consisting of all possible elements

Ex: Set of all cities having Five Star Hotel

Ex. Set of all cities having Seven Star Hotel

- Classical or Crisp Set is a set with fixed and well-defined boundary

Representation of Crisp Set

- $A=\{a_1,a_2,.....,a_n\}$
- $A=\{x|P(x)\}$, P: property
- **Using characteristic function**

$$\mu_A(x)=\begin{cases}1, & \text{if x belongs to A,}\\ 0, & \text{if x does not belong to A.}\end{cases}$$

Boolean Algebra : For discrete set

$U=\{1,2,3,4,5,6\}\quad A=\{3,4,5\}\quad B=\{1,5\}$

$A\cup B=\{1,3,4,5\}\quad A\cap B=\{5\}\quad A^c=\{1,2,6\}$

Fuzzy Set

- Sets with imprecise/vague boundaries
- Potential tool for handling imprecision and uncertainties
- Fuzzy set is a more general concept of the classical set

Examples:

'He is a wealthy man' or **'The car is quite expensive'** or **'Old people are weak'**

While expressions like rich, very, expensive, old, weak, and so on are extremely useful in practical communication, they are inexact in the sense that there are no well-defined boundaries **between wealthy and poor, expensive and cheap, old and young, or weak and strong.**

Fuzzy Algebra : For fuzzy Set

$$F = (s, \mu(s)) | s \in X \text{ and } \mu(s) \text{ is the degree of } s.$$

Example:

X= Cities in Maharashtra

A = Big Cities

A={(Mumbai, 1.0), (Nagpur, 0.8), (Pune, 0.7), (Nashik, 0.2), (Kolhapur, 0.45), (Nagar,0), (Sangli,0.1)}

City	Mumbai	Nagpur	Pune	Nashik	Kolhapur	Nagar	Sangli
μ	1.0	0.8	0.7	0.2	0.45	0	0.1

Let U be the universe of discourse, A and B be two fuzzy subsets of U, and $\overline{A}$ be the complement of A relative to U. Also, let u be an element of U. Then,

$$\begin{aligned} \mu_{\overline{A}}(u) &= 1 - \mu_A(u) \\ \mu_{A \cup B}(u) &= max(\mu_A(u), \mu_B(u)) \\ \mu_{A \cap B}(u) &= min(\mu_A(u), \mu_B(u)) \end{aligned}$$

Let us consider the following two fuzzy sets:

$$A(x) = \{(x_1, 0.1), (x_2, 0.2), (x_3, 0.3), (x_4, 0.4)\}$$

$$B(x) = \{(x_1, 0.5), (x_2, 0.7), (x_3, 0.8), (x_4, 0.9)\}$$

$$\mu_{(A \cup B)}(x) = \max\{\mu_A(x), \mu_B(x)\}$$

$$= \{(x_1, 0.5), (x_2, 0.7), (x_3, 0.8), (x_4, 0.9)\}$$

$$\mu_{(A \cap B)}(x) = \min\{\mu_A(x), \mu_B(x)\}$$

$$= \{(x_1, 0.1), (x_2, 0.2), (x_3, 0.3), (x_4, 0.4)\}$$

Complement $\bar{A}(x) = \{(x_1, 0.9), (x_2, 0.8), (x_3, 0.7), (x_4, 0.6)\}$

Fuzzy Information Retrieval

- Fuzzy sets are modeled based on a **thesaurus**
- This thesaurus is built as follows:
- Let $\bar{C}$ **term-term correlation** matrix
- Let $\mathbf{c_{i,l}}$ be a normalized correlation factor between two terms $\mathbf{k_i}$ and $\mathbf{k_l}$:
- Index terms are not independnet and dependency is there.

$$c_{i,l} = \frac{n_{i,l}}{n_i + n_l - n_{i,l}}$$

- n_i: number of docs which contain k_i
- n_l: number of docs which contain k_l
- $n_{i,l}$: number of docs which contain both k_i and k_l

Document	Content
D1	Shipment of gold damaged in a fire
D2	Delivery of silver arrived in a silver truck
D3	Shipment of gold arrived in a truck
Query	gold silver truck
Keywords	shipment, gold, damaged, fire, delivery, silver, arrived, truck,

Term-Term ($\bar{c}$)Correlation Matrix

	Arrived	Damaged	Delivery	Fire	Gold	Silver	Shipment	Truck
Arrived	1	0	1/2	0	1/3	1/2	1/3	2/2
Damaged	0	1	0	1/1	1/2	0	1/2	0
Delivery			1	0	0	1	0	1/2
Fire				1	1/2	0	1/2	0
Gold					1	0	1	1/3
Silver						1	0	1/2
Shipment							1	1/3
Truck								1

Document	Content
D1	Shipment of gold damaged in a fire
D2	Delivery of silver arrived in a silver truck
D3	Shipment of gold arrived in a truck
Query	gold silver truck
Keywords	shipment, gold, damaged, fire, delivery, silver, arrived, truck,

Query	gold silver truck
Keywords	shipment, gold, damaged, fire, delivery, silver, arrived, truck,

	Gold	Silver	Truck
Arrived	1/3	1/2	2/2
Damaged	1/2	0	0
Delivery	0	1	1/2
Fire	1/2	0	0
Gold	1	0	1/3
Silver	0	1	1/2
Shipment	1	0	1/3
Truck	1/3	1/2	1

- The correlation factor $c_{i,l}$ can be used to define fuzzy set membership for a document d_j as follows:

$$\mu_{i,j} = 1 - \prod_{k_l \epsilon\, d_j} 1 - c_{i,l}$$

The above expression computes an algebric sum over all terms in $\mathbf{d_j}$

$(1 - c_{i,l})$ is the fraction of documents containing one of term i and term l but not both

Document 1

$c_{i,l}$

Query(i)\Keywords(l)	shipment	gold	damaged	fire
gold	1	1	1/2	1/2
silver	0	0	0	0
truck	1/3	1/3	0	0

Query (i)\Keywords(l)	shipment	gold	damaged	fire	$\prod_{k_l \epsilon\, d_j}$	$1 - \prod_{k_l \epsilon\, d_j} 1 - c_{i,l}$
gold	(1-1)	(1-1)	(1-1/2)	(1-1/2)	0	1
silver	(1-0)	(1-0)	(1-0)	(1-0)	1	0
truck	(1-1/3)	(1-1/3)	(1-0)	(1-0)	4/9	5/9

$$\mu_{i,j} = 1 - \prod_{k_l \in d_j} 1 - c_{i,l}$$

The above expression computes an algebric sum over all terms in $\mathbf{d_j}$

Document 2

($1 - c_{i,l}$) is the fraction of documents containing one of term i and term l but not both

$c_{i,l}$

Query(i)\Keywords(l)	delivery	silver	arrived	truck
gold	0	0	1/3	1/3
silver	1	1	1	1/2
truck	1/2	1/2	1	1

Query (i)\Keywords(l)	delivery	silver	arrived	truck	$\prod_{k_l \in d_j}$	$1 - \prod_{k_l \in d_j} 1 - c_{i,l}$
gold	(1-0)	(1-0)	(1-1/3)	(1-1/3)	4/9	5/9
silver	(1-1)	(1-1)	(1-1)	(1-1/2)	0	1
truck	(1-1/2)	(1-1/2)	(1-1)	(1-1)	0	1

$$\mu_{i,j} = 1 - \prod_{k_l \epsilon\, d_j} 1 - c_{i,l}$$

The above expression computes an algebric sum over all terms in $\mathbf{d_j}$

$(1 - c_{i,l})$ is the fraction of documents containing one of term i and term l but not both

Document 3

$c_{i,l}$

Query(i)\Keywords(l)	shipment	gold	arrived	truck
gold	1	1	1/3	1/3
silver	0	0	1/2	1/2
truck	1/3	1/3	1	1

Query (i)\Keywords(l)	s shipment	gold	arrived	truck	$\prod_{k_l \epsilon\, d_j}$	$1 - \prod_{k_l \epsilon\, d_j} 1 - c_{i,l}$
gold	(1-1)	(1-1)	(1-1/3)	(1-1/3)	0	1
silver	(1-0)	(1-0)	(1-1/2)	(1-1/2)	1/4	3/4
truck	(1-1/3)	(1-1/3)	(1-1)	(1-1)	0	1

Document	Content
D1	Shipment of gold damaged in a fire
D2	Delivery of silver arrived in a silver truck
D3	Shipment of gold arrived in a truck

Documents\Query	gold	silver	truck
D1	**1**	**0**	**5/9**
D2	**5/9**	**1**	**1**
D3	**1**	**3/4**	**1**

1) Query: Gold **AND** Silver

Use Min opertor (0,5/9,3/4) Ranking **D3,D2,D1**

2) Query: Silver **AND** Truck

Use Min opertor (0,1,3/4) Ranking **D2,D3,D1**

3) Query: Gold **AND** Truck

Use Min opertor (5/9, 5/9,1) Ranking **D3,D1,D2**

4) Query: Gold **AND** Silver **AND** Truck

Use Min opertor (0,,5/9,3/4) Ranking **D3,D2,D1**

5) Query: Gold **OR** Silver

Use Max operator (1,1,1) Ranking **D1,D2,D3**

6) Query: Gold **AND** (Silver **OR** Truck)

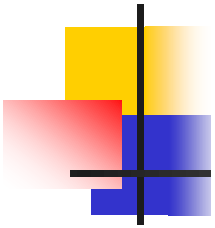
(Gold **AND** Silver) **OR** (Gold **AND** Truck)

Advantages:

- Non-binary index term assignment to documents is supported.
- It is possible to find relevant materials that do not strictly satisfy the query.
- Sets of ranked results

Disadvantages:

- Fuzzy IR models have primarily been discussed in the fuzzy theory literature.
- There are no experiments with standard test collections available.
- It's difficult to compare at the moment.
- Inadequate query processing ability



Exercise 2

Document	Content
D1	step man mankind
D2	step man china
D3	step mankind

	step	man	mankind	china
step	1	0.67	0.67	0.33
man		1	0.33	0.5
mankind			1	0
china				1

$$\mu_{i,j} = 1 - \prod_{k_l \epsilon d_j} 1 - c_{i,l}$$

Document 1

$c_{i,l}$

Query(i)\Keywords(l)	step	man	mankind
step	1	0.67	0.67
man	0.67	1	0.33
mankind	0.67	0.33	1
china	0.33	0.5	0

Query (i)\Keywords(l)	step	man	mankind	$\prod_{k_l \epsilon d_j}$	$1 - \prod_{k_l \epsilon d_j} 1 - c_{i,l}$
step	(1-1)	(1-0.67)	(1-0.67)	0	1
man	(1-0.67)	(1-1)	(1-0.33)	0	1
mankind	(1-0.67)	(1-0.33)	(1-1)	0	1
china	(1-0.33)	(1-0.5)	(1-0)	0.335	0.665

$$\mu_{i,j} = 1 - \prod_{k_l \epsilon d_j} 1 - c_{i,l}$$

Document 2

$c_{i,l}$

Query(i)\Keywords(l)	step	man	china
step	1	0.67	0.33
man	0.67	1	0.5
mankind	0.67	0.33	0
china	0.33	0.5	1

Query (i)\Keywords(l)	step	man	china	$\prod_{k_l \epsilon d_j}$	$1 - \prod_{k_l \epsilon d_j} 1 - c_{i,l}$
step	(1-1)	(1-0.67)	(1-0.33)	0	1
man	(1-0.67)	(1-1)	(1-0.5)	0	1
mankind	(1-0.67)	(1-0.33)	(1-0)	0.22	0.78
china	(1-0.33)	(1-0.5)	(1-1)	0	1

Document	Content
D1	step man mankind
D2	step man china
D3	step mankind

Documents\Query	step	man	mankind	china
D1	**1**	**1**	**1**	**0.665**
D2	**1**	**1**	**0.78**	**1**
D3	**1**	**0.78**	**1**	**.33**

1) Query: step **AND** man

2) Query: step **OR** man

3) Query: step **AND** man **AND** makind

4) Query: man **OR** mankind **OR** china

5) Query: step **AND** (china **OR** mankind)

6) Query: man **AND** (step **OR** china)

Extended Boolean Model

- The Boolean model is **easy to understand.**
- However, there is **no provision for ranking.**
- Also, **for conjunctive queries**, if documents contain **either term**, they are treated the same way as **irrelevant documents**.
- Like the fuzzy model, a ranking can be achieved by e**asing the condition on set membership.**
- It adds the concepts of **partial matching and word weighting** to the **Boolean model**. Therefore name is **Extended Boolean Model.**
- Combine **Vector model characteristics** with **Boolean algebra properties**.

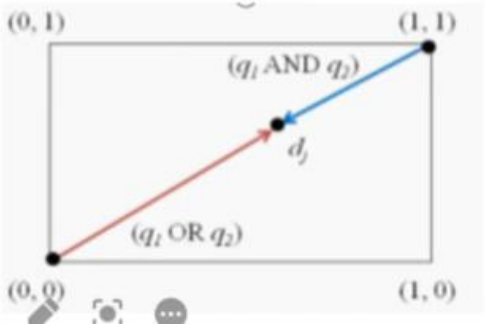
- The concept is **quite simple**.
- **Keywords** are **weighted (as in the vector space model)** but users can utilise **AND and OR in the searches (as in the Boolean model).**
- Documents are **ranked by similarity fucntion as in Vector Model**
- In the Extended Boolean Model, **"keywords local and global weights" are first normalised by their highest scores**, and then **their products are calculated**.

Formal characterization Extended Boolean Model

Terms	Notation
Normalized Local Weight for each Document.	$L_{i,j} = \frac{f_{i,j}}{max\, f_{i,j}}$
Global Weight Across all Documents	$IDF_i = log\frac{N}{n_i}$ or $IDF_i = log(1 + \frac{N}{n_i})$
Normalized Global Weight Across all Documents	$G_i = \frac{IDF_i}{max\, IDF_i}$
Document Weight w.r.t. keyword	$w_{i,j} = L_{i,j} \times G_i$

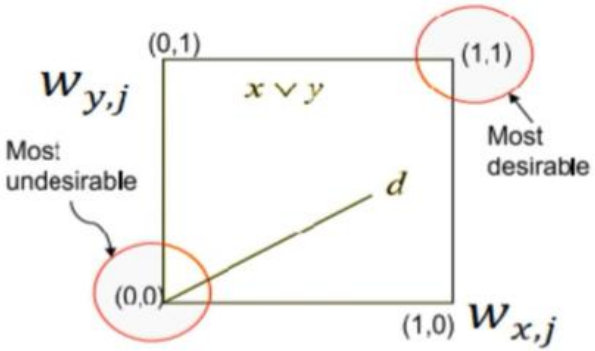
Keywords Space Representation for AND and OR operators for Two Term queries

- For **AND queries**, the (1, 1) point represents the case where both terms are present in a document.
- For **OR queries**, (0, 0) point represents the case where both terms are absent from a document.
- Thus, document term weights adopt values between 0 and 1. For a document j mentioning two terms, $w_{1,j}$ and $w_{2,j}$ the term assignment can be described by a two-dimensional term space.

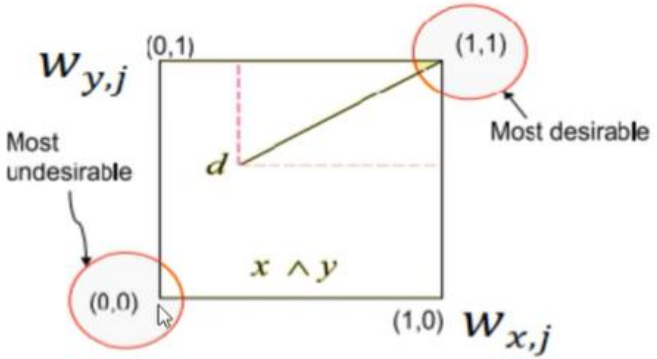


- The maximum Euclidean distance, dist max between the (0, 0) and (1, 1) points is

$$Dist_{max} = \sqrt{(1-0)^2 + (1-0)^2} = \sqrt{2}$$



(a) $x \vee y$.



(b) $x \wedge y$.

AND Query

$$dist_{AND} = \sqrt{2} - \sqrt{(1 - w_{1,j})^2 + (1 - w_{2,j})^2}$$

$$sim(d_j, q_{AND}) = \frac{dist_{j,AND}}{dist_{max}} = 1 - \sqrt{\frac{(1 - w_{1,j})^2 + (1 - w_{2,j})^2}{2}}$$

OR Query

$$dist_{j,OR} = \sqrt{(w_{1,j} - 0)^2 + (w_{2,j} - 0)^2}$$

$$sim(d_j, q_{OR}) = \frac{dist_{j,OR}}{dist_{max}} = \sqrt{\frac{(w_{1,j} - 0)^2 + (w_{2,j} - 0)^2}{2}}$$

		Dissimilaity (Distance)	Similarity
a and b query term		AND	AND
0	0	0	0
0	1	0.41	0.29
1	0	0.41	0.29
1	1	1.41	1

		Dissimilaity (Distance)	Similarity
a or b query term		OR	OR
0	0	0	0
0	1	1	0.71
1	0	1	0.71
1	1	1.41	1

AND Query

$$dist_{AND} = \sqrt{2} - \sqrt{(1 - w_{1,j})^2 + (1 - w_{2,j})^2}$$

$$sim(d_j, q_{AND}) = \frac{dist_{j,AND}}{dist_{max}} = 1 - \sqrt{\frac{(1 - w_{1,j})^2 + (1 - w_{2,j})^2}{2}}$$

OR Query

$$dist_{j,OR} = \sqrt{(w_{1,j} - 0)^2 + (w_{2,j} - 0)^2}$$

$$sim(d_j, q_{OR}) = \frac{dist_{j,OR}}{dist_{max}} = \sqrt{\frac{(w_{1,j} - 0)^2 + (w_{2,j} - 0)^2}{2}}$$

for n numbers of query terms using Lp-norms, where $1 \leq p \leq \infty$.

$$sim(d_j, q_{AND}) = 1 - \left(\frac{(1-w_{1,j})^p + (1-w_{2,j})^p + \cdots + (1-w_{n,j})^p}{n}\right)^{1/p}$$

$$sim(d_j, q_{OR}) = \left(\frac{w_{1,j}{}^p + w_{2,j}{}^p + \cdots + w_{n,j}{}^p}{n}\right)^{1/p}$$

For two term query: n=2, p=2

For three term query: n=3, p=2 [If Euclidean Distance]

- For instance, consider the query $q=(k_1 \wedge k_2) \vee k_3$. The similarity $sim(q,d_j)$ between a document d_j and this query is then computed as

$$sim(q,d) = \left(\frac{\left(1-\left(\frac{(1-x_1)^p+(1-x_2)^p}{2}\right)^{1/p}\right)^p + x_3^p}{2}\right)^{1/p}$$

The query $q=k_1$ AND k_2 OR k_3 can be evaluated recursively by

1) k_1 AND k_2 part

2) Resultant of first OR k_3

Advantages:

- A hybrid model included properties of both set theoretic and algebric models
- Model is quite powerful
- By varying parameter p, we get ranking behaviour from vector model to fuzzy model

Disadvantage

- Computation is complex.
- Assume mutul independence of index terms
- Distributivity operation does not hold for ranking computation

$$q_1 = (k_1 \vee k_2) \wedge k_3$$
$$q_2 = (k_1 \wedge k_3) \vee (k_2 \wedge k_3)$$
$$sim(q_1, d_j) \neq sim(q_2, d_j)$$

Exercise 1

Document	Content
D1	Computer Information Retrieval
D2	Computer Retrieval
D3	Information
D4	Computer Information

Step 1: Stemming & Stop Word Removal

Step 2: Normalized Local Weight for each Document

Step 3: Normalized Global Weight Across all Documents

Step 4: Document weight= Normalized Local × Normalized Global

Step 5: Depending on no. of terms in query apply similarity function

Sr. No.	Index Term or Keywords	Doc1	Doc2	Doc3	Doc4	Count	Query: Computer & Retrieval	IDF_i	G_i
1	Computer	1	1	0	1	3	1	0.37	0.8
2	Information	1	0	1	1	3	0	0.37	0.8
3	Retrieval	1	1	0	0	2	1	0.48	1.0
	Max Frequency	1	1	1	1		1		

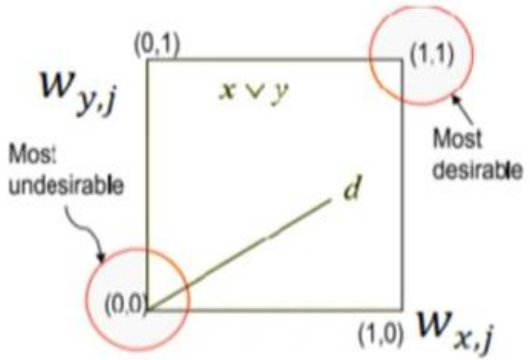
$$L_{i,j} = \frac{f_{i,j}}{max\, f_{i,j}}$$

$$IDF_i = log(1 + \frac{N}{n_i})$$

$$G_i = \frac{IDF_i}{max\, IDF_i}$$

Sr. No.	Index Term or Keywords	Doc1	Doc2	Doc3	Doc4	Query: Computer & Retrieval
1	Computer	0.8	0.8	0.0	0.8	0.8
2	Information	0.8	0.0	0.8	0.8	0.0
3	Retrieval	1.0	1.0	0.0	0.0	1.0

$$w_{i,j} = L_{i,j} \times G_i$$

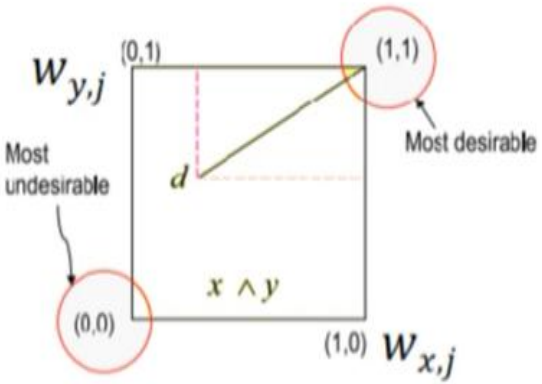


(a) $x \vee y$.

	Computer ($w_{1,j}$)	Retrieval ($w_{2,j}$)	$dist_{AND}$	$sim(d_j, q_{AND})$
Doc1	0.8	1	1.21	0.86
Doc2	0.8	1	1.21	0.86
Doc3	0	0	0.00	0.00
Doc4	0.8	0	0.39	0.28

$$dist_{AND} = \sqrt{2} - \sqrt{(1 - w_{1,j})^2 + (1 - w_{2,j})^2}$$

$$sim(d_j, q_{AND}) = \frac{dist_{j,AND}}{dist_{max}} = 1 - \sqrt{\frac{(1 - w_{1,j})^2 + (1 - w_{2,j})^2}{2}}$$



	Computer ($w_{1,j}$)	Retrieval ($w_{2,j}$)	$dist_{j,OR}$	$sim(d_j, q_{OR})$
Doc1	0.8	1	1.28	0.91
Doc2	0.8	1	1.28	0.91
Doc3	0	0	0.00	0.00
Doc4	0.8	0	0.80	0.57

$$dist_{j,OR} = \sqrt{(w_{1,j} - 0)^2 + (w_{2,j} - 0)^2}$$

$$sim(d_j, q_{OR}) = \frac{dist_{j,OR}}{dist_{max}} = \sqrt{\frac{(w_{1,j} - 0)^2 + (w_{2,j} - 0)^2}{2}}$$

Document	Content
D1	Computer Information Retrieval
D2	Computer Retrieval
D3	Information
D4	Computer Information

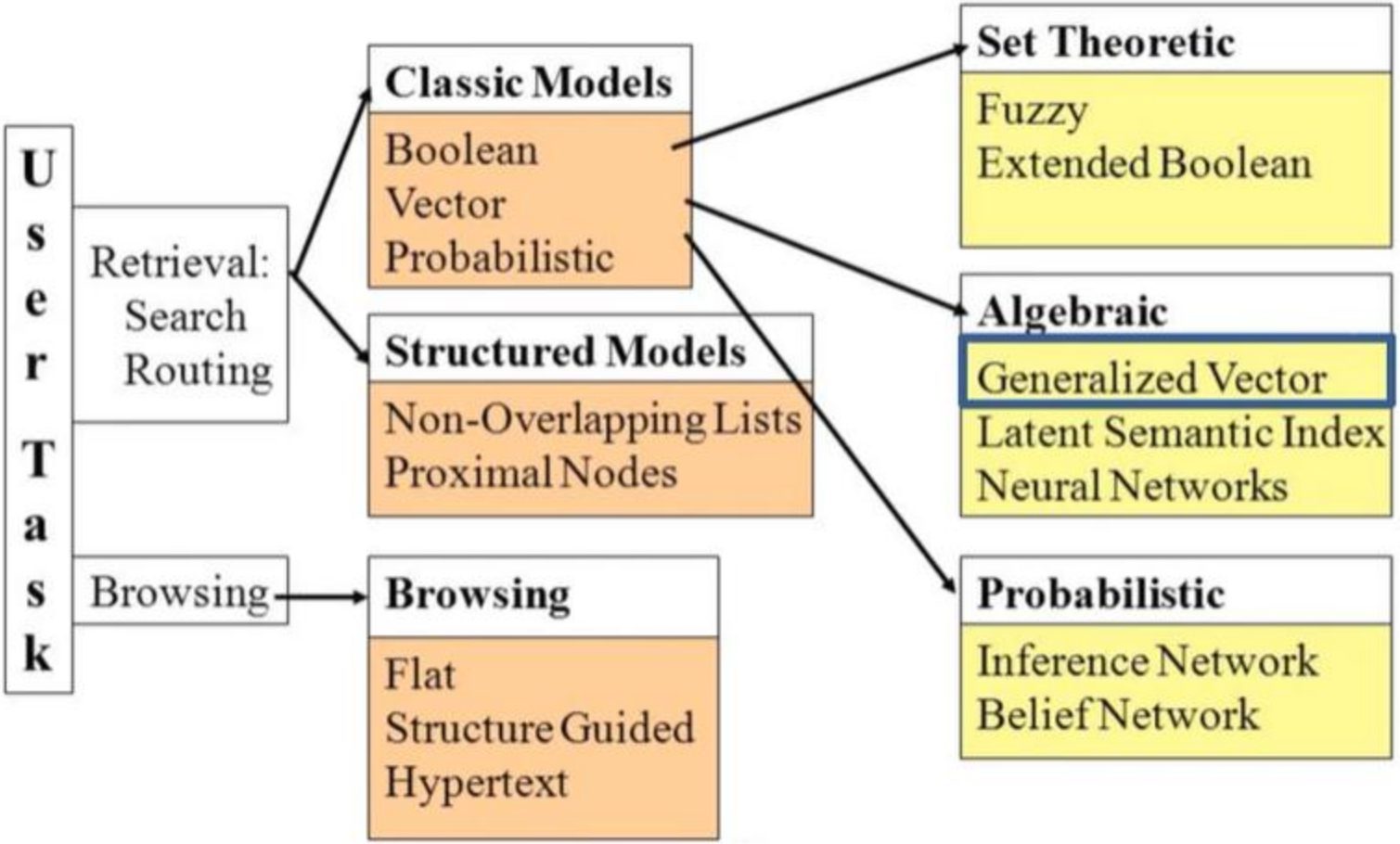
Query: Computer AND Retrieval

	$sim(d_j, q_{AND})$
Doc1	0.86
Doc2	0.86
Doc3	0.00
Doc4	0.28

Query: Computer OR Retrieval

	$sim(d_j, q_{OR})$
Doc1	0.91
Doc2	0.91
Doc3	0.00
Doc4	0.57

Taxonomy of Information Retrieval Models



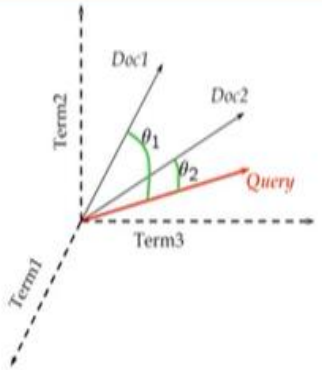
Introduction

The Generalized Vector Model of Information Retrieval (GVMIR) extends the traditional Vector Space Model (VSM) by relaxing the assumption that index term vectors are **linearly independent, while still not requiring them to be pairwise orthogonal.**

This leads to the development of the Generalized Vector Space Model, which is designed to provide a more flexible and realistic representation of documents and queries in information retrieval.

- Classic models enforce independence of index terms.
- For the Vector model:
- Set of term vectors $\{k_1, k_2, ..., k_t\}$ are linearly independent and form a basis for the subspace of interest.
- Frequently, this is interpreted as:
- $V\ i,j\ k_i \bullet kj = 0$

vector similarity: cosine



- In 1985, Wong, Ziarko, and Wong proposed an interpretation in which the set of terms is linearly independent, **but not pairwise orthogonal.**

In the generalized vector model, two index terms might be non-orthogonal and are represented in terms of smaller components (minterms).

Let $w_{i,j}$ be the weight associated with $[k_i, d_j]$

$(k_1, k_2, \ldots, k_t)$ be the set of all terms

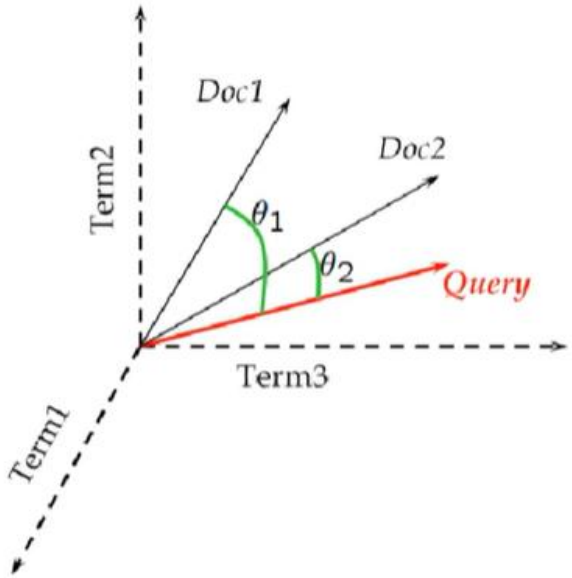
If these weights are all binary, all patterns of occurrence of terms within documents can be represented by the minterms:

- For e.g. Three keyterms are there k_1, k_2 and k_3
- **m1 = (0, 0, 0)** **m2 = (0, 0, 1)** **m3= (0, 1, 0)** **m4= (0, 1, 1)**
- **m5 =(1, 0, 0)** **m6 = (1, 0, 1)** **m7=(1, 1, 0)** **m8=(1, 1, 1)**
- Total terms are m^{2^t}

Main idea: Generalized Vector Model

- Minterm vectors are pairwise orthogonal. But, this does not mean that the index terms are independent:
- The minterm **m7 and m8** is given by: **m7=(1, 1, 0)** **m8=(1, 1, 1)**
- This minterm **m7** indicates the occurrence of the terms **k1 and k2** within a same document for **m7** and minterm **m8** indicates the occurrence of the terms **k1, k2 and k3** within a same document for **m8**.
- If such document exists in a collection, the minterm m7 is active and that a dependency between these two terms is induced. **(For m8, dependency between three terms)**
- The generalized vector model adopts as a basic foundation the notion that cooccurence of terms within documents induces dependencies among them.

Vector Model



Term1, Term2, Term3 vectors are

(1, 0, 0) (0, 1, 0) and (0, 0, 1).

All terms (keywords) are independent.

Term=3

Generalized Vector Model

m1 = (0, 0, 0) m2 = (0, 0, 1) m3= (0, 1, 0) m4= (0, 1, 1)

m5 =(1, 0, 0) m6 = (1, 0, 1) m7=(1, 1, 0) m8=(1, 1, 1)

	1	2	3	4	5	6	7	8
$\overrightarrow{m1}$	0	0	0	0	0	0	0	0
$\overrightarrow{m2}$	0	1	0	0	0	0	0	0
$\overrightarrow{m3}$	0	0	1	0	0	0	0	0
$\overrightarrow{m4}$	0	0	0	1	0	0	0	0
$\overrightarrow{m5}$	0	0	0	0	1	0	0	0
$\overrightarrow{m6}$	0	0	0	0	0	1	0	0
$\overrightarrow{m7}$	0	0	0	0	0	0	1	0
$\overrightarrow{m8}$	0	0	0	0	0	0	0	1

- Minterm vectors are pairwise orthogonal.
- But, this does not mean that the index terms are independent. Each minterm specifies a kind of dependence among index terms
- That is, the co-occurrence of index terms inside docs in the collection induces dependencies among these index terms

Example:

Total 5 documents D1, D2, D3, D4 and D5.

Two keyterms : K1 and K2

Query (q): K1=1, K2=5

Document	K1	K2
D1	1	2
D2	4	0
D3	0	5
D4	0	2
D5	3	0

-

Example:

Document	K1	K2	Minterm
D1	1	2	m1
D2	4	0	m2
D3	0	5	m3
D4	0	2	m3
D5	3	0	m2

Minterm	K1	K2
m1	1	1
m2	1	0
m3	0	1
m4	0	0

Document	K1	K2	Minterm	Minterm vector
D1	1	2	m1	$\overrightarrow{m1}$ =(1, 0, 0, 0)
D2,D5	7	0	m2	$\overrightarrow{m2}$=(0, 1, 0, 0)
D3 ,D4	0	7	m3	$\overrightarrow{m3}$=(0, 0, 1, 0)
Query (q)	1	5	m1	

The weight $c_{i,r}$ associated with the pair $[k_i, m_r]$ sums up the weights of the term k_i in all the documents which have a term occurrence pattern given by m_r.

$$c_{i,r} = \sum_{d_j \mid g_l(\vec{d_j})=g_l(m_r) \text{ for all } l} w_{i,j}$$

The vector associated with the term k_i is computed as

$$\vec{k}_i = \frac{\sum_{\forall r,\ g_i(m_r)=1} c_{i,r}\ \vec{m}_r}{\sqrt{\sum_{\forall r,\ g_i(m_r)=1} c_{i,r}^2}}$$

$c_{i,r}$:

i - term (keyword) K1 and K2

r- minterm (m1, m2, m3)

i=1, 2

r=1, 2, 3

Document	K1	K2	Minterm	Minterm vector
D1	1	2	m1	$\overrightarrow{m1}$ =(1, 0, 0, 0)
D2,D5	7	0	m2	$\overrightarrow{m2}$=(0, 1, 0, 0)
D3 ,D4	0	7	m3	$\overrightarrow{m3}$=(0, 0, 1, 0)
Query (q)	1	5	m1	

c1,1	c1,2	c1,3
1	7	0
c2,1	**c2,2**	**c2,3**
2	0	7

Prepare two vectors k1 and k2

c1,1	c1,2	c1,3
1	7	0
c2,1	**c2,2**	**c2,3**
2	0	7

$$k1 = \frac{m1 + 7m2 + 0m3}{\sqrt{1^2 + 7^2 + 0^2}} = \frac{m1 + 7m2}{\sqrt{50}}$$

$$k2 = \frac{2m1 + 0m2 + 7m3}{\sqrt{2^2 + 0^2 + 7^2}} = \frac{2m1 + 7m3}{\sqrt{53}}$$

$\overrightarrow{m1}$ =(1, 0, 0, 0)
$\overrightarrow{m2}$=(0, 1, 0, 0)
$\overrightarrow{m3}$=(0, 0, 1, 0)

$$D1 = k1 + 2k2 \qquad q = k1 + 5k2$$

$$D1 = \frac{m1}{\sqrt{50}} + \frac{7m2}{\sqrt{50}} + \frac{4m1}{\sqrt{53}} + \frac{14m3}{\sqrt{53}} \qquad q = \frac{m1}{\sqrt{50}} + \frac{7m2}{\sqrt{50}} + \frac{10m1}{\sqrt{53}} + \frac{35m3}{\sqrt{53}}$$

$$D1 = \left(\frac{1}{\sqrt{50}} + \frac{4}{\sqrt{53}}\right)m1 + \frac{7m2}{\sqrt{50}} + \frac{14m3}{\sqrt{53}} \qquad D1 = 0.69m1 + .99m2 + 1.92m3$$

$$q = \left(\frac{1}{\sqrt{50}} + \frac{10}{\sqrt{53}}\right)m1 + \frac{7m2}{\sqrt{50}} + \frac{35m3}{\sqrt{53}} \qquad q = 1.52m1 + .99m2 + 4.81m3$$

The dot product as follows:

$D1 \cdot q = (0.69m1 + 0.99m2 + 1.92m3) \cdot (1.52m1 + 0.99m2 + 4.81m3)$

$D1 \cdot q = (0.69 * 1.52)m1 \cdot m1 + (0.99 * 0.99)m2 \cdot m2 + (1.92 * 4.81)m3 \cdot m3$

$D1 \cdot q = 1.0476m1 \cdot m1 + 0.9801m2 \cdot m2 + 9.2352m3 \cdot m3$

Since the basis vectors are orthogonal ($m_i \cdot m_j = 0$ for $i \neq j$) and have a length of 1 ($m_i \cdot m_i = 1$), the dot product simplifies to:

$D1 \cdot q = 1.0476 * 1 + 0.9801 * 1 + 9.2352 * 1$

$D1 \cdot q = 1.0476 + 0.9801 + 9.2352$

D1.q = 11.2629

Ranking

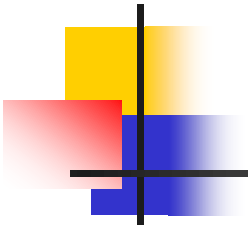
Document	K1	K2	Ranking
D1	1	2	**11.2629**
D2	4	0	**4.6779**
D3	0	5	**25.2185**
D4	0	2	**10.0712**
D5	3	0	**3.5757**
Query (q)	1	5	

Latent Semantic Analysis

- When we use index terms (keywords) for searching:
- Problem 1: Many unrelated documents may appear (because they share the same keyword, but not the same meaning).
- Problem 2: Some relevant documents may not appear (because they use different words to express the same concept).
- Instead of matching documents strictly by **keywords**, **LSI** tries to match them by **concepts**.
- Words that appear in similar contexts are assumed to have similar meanings.
- A document can be considered relevant **even if it doesn't contain the exact keyword** from the query, as long as it shares the same concept.

How LSI Works

- Construct a term-document matrix (M):
- Rows → terms (keywords)
- Columns → documents
- Each entry Mi,jMi,j → weight showing how important term i is in document j.
- Weight often calculated using TF-IDF (term frequency–inverse document frequency).
- Apply Dimensionality Reduction (using Singular Value Decomposition, SVD).
- This reduces the large term-document matrix into a smaller concept space.
- Each document and query is mapped into this lower-dimensional space.Retrieval happens in this concept space instead of raw keyword space.



- **Advantages**
- Captures **synonymy** (different words, same meaning → car/automobile).
- Handles **polysemy** better (same word, multiple meanings → “bank” as riverbank or financial bank).
- Improves retrieval accuracy compared to plain keyword matching.

1. Decomposition of the Term-Document Matrix

We start with the **term-document matrix** M (terms × documents).

LSI applies **Singular Value Decomposition (SVD)** to break M into three matrices:

$$M = KSD^t$$

- K → Matrix of **eigenvectors** from the term-term correlation (MM^t)
- D^t → Matrix of **eigenvectors** from the document-document correlation (M^tM)
- S → Diagonal matrix of **singular values** (importance weights of concepts)

This decomposition transforms raw keyword space into a **concept space**.

◆ 2. Dimensionality Reduction

- Not all singular values in S are equally important.
- Keep only the **top** s singular values (the most meaningful "concepts") and discard the smaller ones.

This gives a reduced matrix:

$$M_s = K_s S_s D_s^t$$

where $s < r$ (rank).

3. Concept Space

- The parameter s controls the dimensionality of the new semantic space.
- **If s is too large:** Noise and redundancy remain.
- **If s is too small:** Important distinctions may be lost.
- The trick is to balance: enough concepts to capture meaning, but few enough to eliminate noise.

---

4. Similarity Between Documents

The similarity between documents in this reduced concept space is computed as:

$$M_s^t M_s = (D_s S_s)(D_s S_s)^t$$

- The (i, j) entry of this matrix measures the **semantic similarity** between documents d_i and d_j.
- This means documents that share concepts (even if they don't share exact words) are considered related.

5. Query as a Pseudo-Document

- A user query is treated as a **pseudo-document** in the term-document matrix.
- Example: Suppose query = "car". We model it like an extra document (say, document 0).
- Then compute similarity between this pseudo-document and all real documents in M_s.
- The result is a **ranking of documents** by conceptual relevance to the query.

---

6. Benefits

- Efficient indexing because $s \ll t$ (terms) and $s \ll N$ (documents).
- Eliminates redundancy and noise.
- Matches **concepts** instead of exact words.

---

7. Limitation

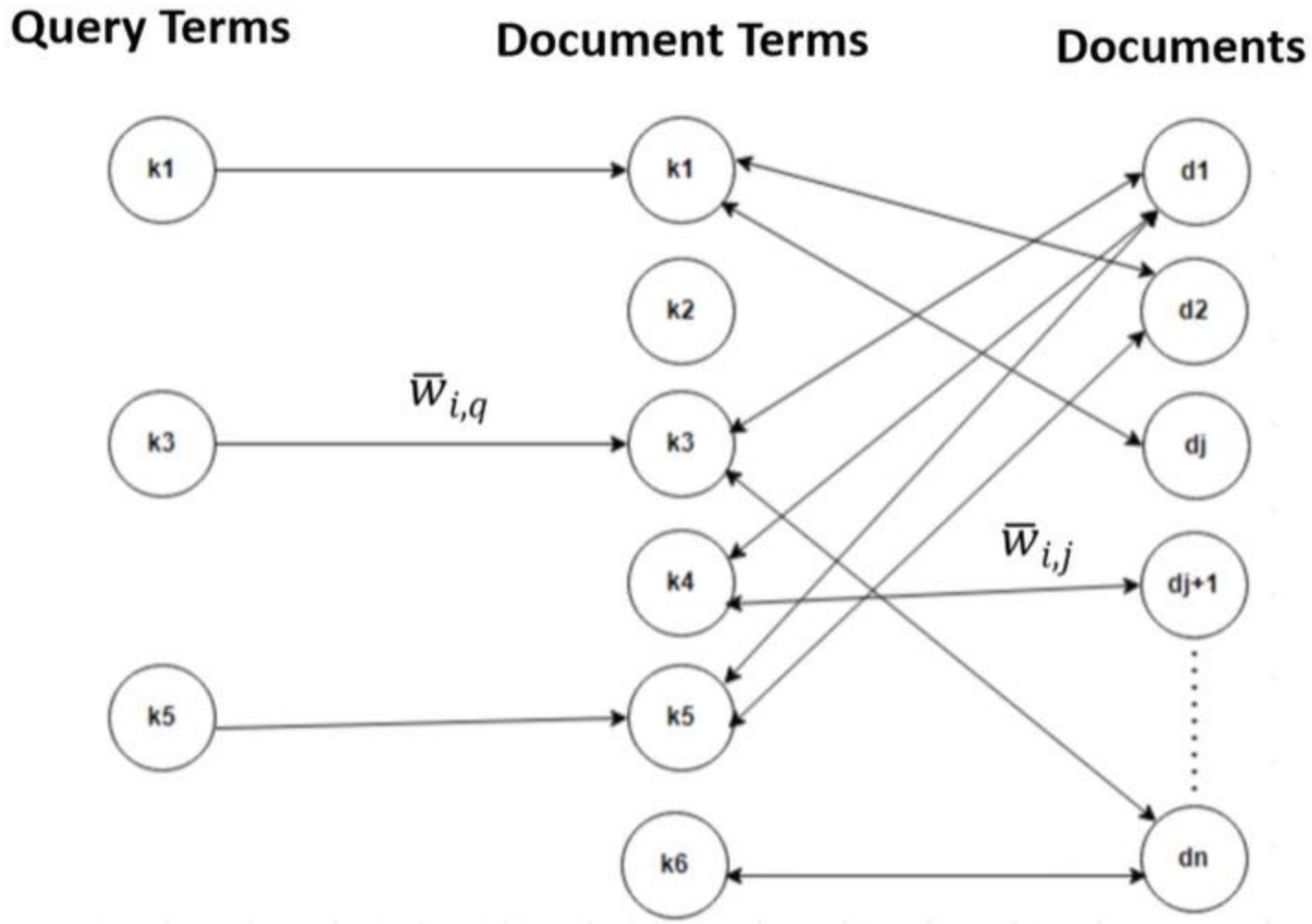
- While mathematically elegant, whether LSI is always **superior in practice** for very large collections still needs empirical validation.

Introduction to Neural Network Model

- Application of Neural Networks as an alternative model for Information Retrieval.
- It outlines the structure of a neural network designed for this purpose, consisting of three layers: **one for query terms, one for document terms, and a third for the documents**.
- In this neural network, the process begins with **query term nodes** initiating the signal propagation to **document term nodes**, followed by the potential generation of signals by **document term nodes** to **document nodes**.
- This **completes the initial phase where signals travel from query term nodes to document nodes.**



- Importantly, the neural network does not halt after this initial phase.
- Document nodes can generate signals directed back to document term nodes through bidirectional edges.
- Document term nodes, in turn, can transmit signals to document nodes, repeating the process with signals weakening at each iteration.
- This process may activate documents even if they lack query terms, resembling the activation of a *built-in thesaurus*.



Neural Network Model for Information Retrieval



Three layers network

Signals propagate across the network

First level of propagation:

- Query terms issue the first signals
- These signals propagate accross the network to reach the document nodes

Second level of propagation:

- Document nodes might themselves generate new signals which affect the document term nodes
- Document term nodes might respond with new signals of their own

Normalize signal strength **(MAX = 1).** Query terms emit initial signal equal to 1.

Weight associated with an edge from a **query term node k_i** to a **document term node k_j**:

$$w_{i,q} = (0.5 + \frac{freq_{i,q}}{maxl\ freq_{i,q}}) \times log\frac{N}{n_i} \qquad \overline{w}_{i,q} = \frac{w_{i,q}}{\sqrt{\sum_{i=1}^{t} w_{i,q}^2}}$$

Weight associated with an edge from a **document term node k_i to a document node d_j**:

$$w_{i,j} = f_{i,j} \times log\frac{N}{n_i} \qquad \overline{w}_{i,j} = \frac{w_{i,j}}{\sqrt{\sum_{i=1}^{t} w_{i,j}^2}}$$

After the first level of signal propagation, the activation level of a document node $\mathbf{d}_j$, is given by: which is exactly the ranking of the Vector model.

$$\sum_{i=1}^{t} w_{i,q}\, w_{i,j} = \frac{\Sigma_{i=1}^{t} w_{i,q} \times w_{i,j}}{\sqrt{\Sigma_{i=1}^{t} w_{i,q}^2} \times \sqrt{\Sigma_{i=1}^{t} w_{i,j}^2}}$$

- New signals might be exchanged among document term nodes and document nodes in a process analogous to a feedback cycle
- A minimum threshold should be enforced **to avoid counterfeit signal generation**

Introduction to Structured Model

- Keyword-based query answering often treats documents as **flat** entities where every word in the document has the **same weight**. This approach doesn't consider the **hierarchical or structured nature of documents.**
- Enhancing Keyword-Based Models: To improve keyword-based models, it's essential to take into account the **document structure**. Different sections of a document may have varying levels of importance. For example, words appearing in the **title or sub-titles** within the document could be given **higher weights,** indicating their significance.
- **Definition:** Retrieval models which combine ***information on the text content*** with ***information on the document structure.***

Consider the following information need:

Retrieve all documents which contain a page in which the string

"atomic holocaust" appears in **italics** in the text surrounding a Figure whose label contains the word **"earth"**

The corresponding structural query could be:

same-page(near("atomic holocaust", Figure(label("earth"))))

while classical IR Model query

'atomic holocaust' and 'earth'

which retrieves all the documents containing **both strings**.

However, this answer contains **many more documents (some may be irrelevant) than desired by this user.**

Retrieve all documents containing a chapter where the term "renewable energy sources" appears in bold, and this term is associated with a Figure labeled "sustainability."

The corresponding query could be structured as follows:

same-chapter(near(bold("renewable energy sources"), Figure(label("sustainability"))))

For structural query: The user might be interested in an advanced interface which simplifies the task of specifying structural query.

Drawbacks

- Difficult to specify the structural query for Naive Users
- An advanced user interface is needed to enter structural query
- Ranking is not established in Structured text retrieval model

Tradeoffss

The more expressive the model, the less efficient is its query evaluation strategy

Two Structured Text Retrieval Models

- Non-Overlapping Lists
- Proximal Nodes

6

Basic Definitions

1) Match Point:

Refers to the position in the text where a sequence of words matches the user's query.

For example, if the user's query is **"atomic holocaust in Hiroshima"** and this sequence appears in **three places within the text of a document**, it's said to contain **three match points.**

2) Region:

Denotes a contiguous portion of the text within a document.

Basic Definitions

3) Node:

- Represents a structural component of the document.
- Nodes have predefined topological properties.
- These properties are known both to the document's author and the user searching the document system.
- Nodes can include chapters, sections, subsections, and other similar structural elements within the document.

1) Model based on Non-Overlapping Lists

Text Division: The **entire text of each document** is divided into **non-overlapping text regions.**

Multiple Lists: Several lists are generated to represent these regions. Each list captures different structural components.

For example

- A list of all chapters within the document.
- A list of all sections in the document.
- A list of all subsections within the document.



Distinct Data Structures: Each of these lists is maintained as a separate and distinct data structure.

Non-overlapping: Text regions within the same list do not overlap with each other.

Overlap between Lists: However, it's important to note that text regions from distinct lists might overlap with each other, as the document's structure can sometimes have elements shared between different structural components.

List 1 - Chapters:

Chapter 1: Introduction

Chapter 2: Literature Review

Chapter 3: Methodology

List 2 - Sections:

Section 1.1: Background

Section 1.2: Research Objectives

Section 2.1: Related Work

Section 2.2: Conceptual Framework

Section 3.1: Data Collection

Section 3.2: Data Analysis

List 3 - Subsections:

Subsection 1.1.1: Historical Context

Subsection 1.1.2: Significance

Subsection 2.1.1: Previous Studies

Subsection 2.1.2: Research Gaps

Subsection 3.1.1: Survey Design

Subsection 3.2.1: Quantitative Analysis

Section 1.1: "Background" from List 2 overlaps with part of Chapter 1.

Section 3.1: "Data Collection" from List 2 overlaps with part of Chapter 3.

These overlaps occur because the boundaries between chapters, sections, and subsections are not rigid and can vary depending on the document's structure.

Text regions in the same list : No overlapping , but distinct list might overlap

To allow searching for index terms(vocabulary) and for text regions, a single inverted file is built in which each structural component stands as an entry in the index. Associated with each entry, there is a list of text regions as a list of occurrences.

Moreover, such a list could be easily merged with the traditional inverted file for the words in the text.

Since the text regions are non-overlapping, the types of queries which can be asked are simple:

(a) select a region which contains a given word (and does not contain other regions)

(b) select a region A which does not contain any other region B (where B belongs to a list distinct from the list for A)

(c) select a region not contained within any other region, etc. explain a), b) and c) cases with example

Sorted Index Word/Keyterm	Block Occurrence
beautiful	4, 12,56,78.....
flowers	3,45,67.....
garden	2,9,89.....
house	23,34.....
many	45,50...
List 1 - Chapters	Chapter 1: Introduction, Chapter 2: Literature Review, Chapter 3: Methodology
List 2 - Sections	Section 1.1: Background, Section 1.2: Research Objectives, Section 2.1: Related Work, Section 2.2: Conceptual Framework, Section 3.1: Data Collection, Section 3.2: Data Analysis
List 3 - Subsections	Subsection 1.1.1: Historical Context, Subsection 1.1.2: Significance, Subsection 2.1.1: Previous Studies, Subsection 2.1.2: Research Gaps, Subsection 3.1.1: Survey, Design, Subsection 3.2.1: Quantitative Analysis

(a) select a region which contains a given word (and does not contain other regions)

(b) select a region A which does not contain any other region B (where B belongs to a list distinct from the list for A)

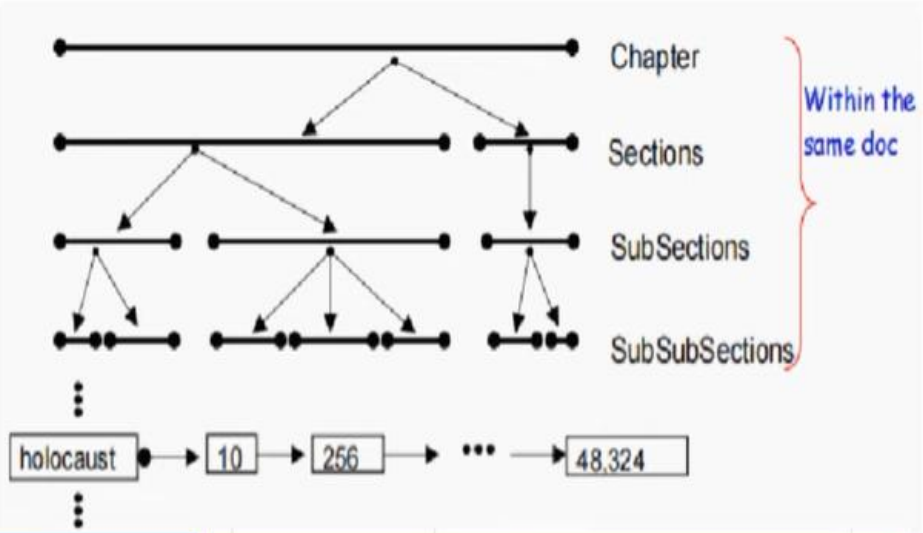
Text regions in the same list: No overlapping , but distinct list might overlap

Advantages and Disadvantages of Non-Overlapping Lists

- The non-overlapping lists model is simple and allows efficient implementation
- But, types of queries that can be asked are limited
- Also, model does not include any provision for ranking the documents by degree of similarity to the query

2) Model based on Proximal Nodes

- The proposed model enables the creation of **hierarchical indexing structures** over the same document text.
- It comprises **chapters, sections, paragraphs, pages, and lines as nodes.**
- Each node is linked to a specific **text region,** enhancing document structure and content retrieval.
- Two distinct hierarchies might refer to overlapping text regions.



Chapter 1: Introduction

Section 1.1: Background

Paragraph 1: Discussing historical context.

Paragraph 2: Presenting key theories.

Section 1.2: Research Objective

Paragraph 1: Defining the research aim.

Paragraph 2: Formulating research questions.

Chapter 2: Literature Review

Section 2.1: Previous Studies

Paragraph 1: Summarizing prior research.

Paragraph 2: Highlighting gaps in knowledge.

Section 2.2: Methodology

Paragraph 1: Explaining research methods.

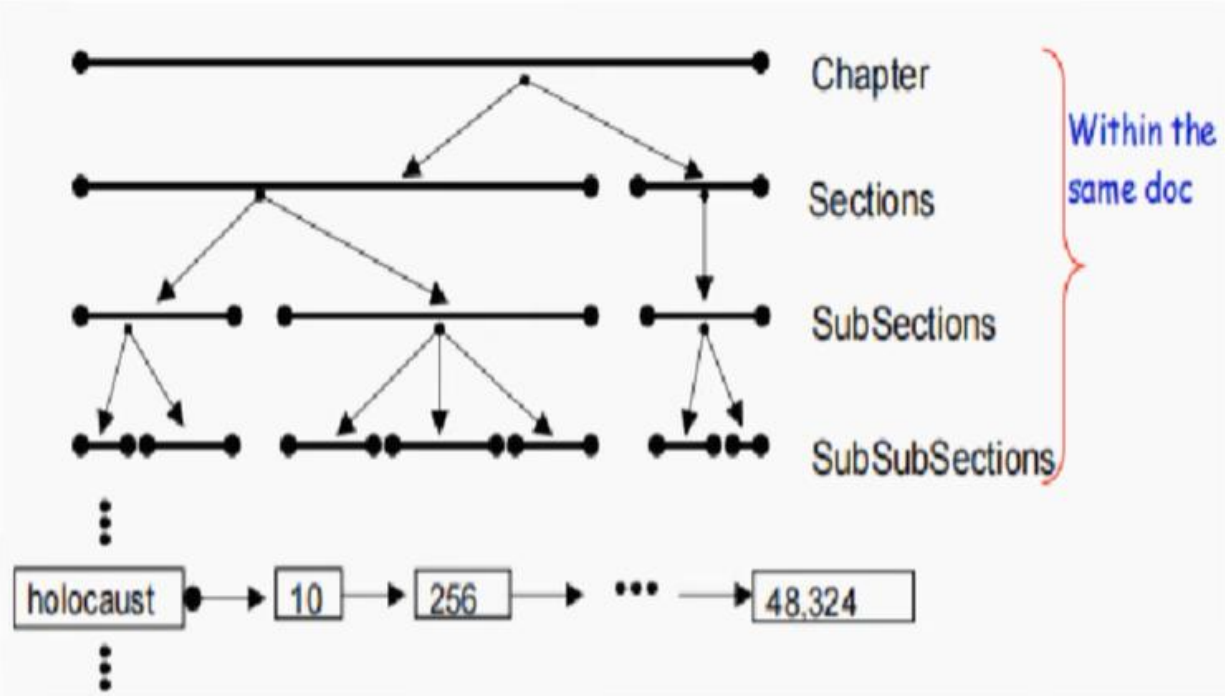
Chapter 3: Findings

Section 3.1: Data Analysis

Paragraph 1: Presenting data.

Paragraph 2: Analyzing results.

- Given a user query which refers to **distinct hierarchies,** the compiled answer is formed by **nodes which all come from only one of them.** Thus, an answer cannot be composed of nodes which come from two distinct hierarchies.
- This approach is designed for **faster query processing, although it sacrifices some expressiveness.**
- For example, let's consider a document that has **two independent hierarchies: one represents chapters and sections, and the other represents pages and paragraphs.**
- If a user query requests information about a specific topic from both hierarchies, a proximal node-based system would allow forming an answer with nodes from one hierarchy only. This means that the response can include from **chapters, sections hierarchy or pages, and paragraphs hierarchy.**



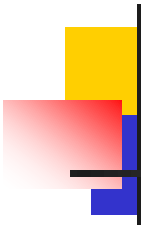
- Single Hierarchy with Four Levels and an inverted list for the word 'holocaust.' The entries in this inverted list indicate all the positions in the text of the document in which the word 'holocaust' occurs.
- In the hierarchy, each node indicates the position in the text of its associated structural component (chapter, section, subsection, or subsubsection).

For e.g. query is [(*section) with ('holocaust')]

- It processes by checking the term's inverted list and looking for corresponding hierarchical sections.
- The query searches for sections, subsections, or subsubsections with the word 'holocaust.'

More Sophisticated Approach

- It processes by checking the term's inverted list and looking for corresponding hierarchical sections.
- First,we search through the document's structure starting from the top (like chapters) and moving down to the bottom (like paragraphs). We will stop when we find a match (i.e., a section that contains the word "holocaust").
- Now, we found mentioned word in in the "innermost" section
- For next word, we dont need to check second occurrence in the inverted file.
- We will check next occurrence of "holocaust" is also within this same section. If it is there it indicates that any larger sections containing this one also have the word. So, we don't need to recheck them.
- We keep doing this for all the occurrences of "holocaust" in the document. This way, we save time because we only look in nearby sections each time, and don't redo the whole search.
- The term "proximal nodes" just means that we're focusing on nearby sections in the document's structure, which makes the search faster.



Advantages and Disadvantages of Proximal Nodes

- The model enables advanced queries compared to non-overlapping lists.
- It speeds up processing by examining nearby nodes.
- Queries are somewhat limited (all nodes in the answer must be from one hierarchy).
- It's an efficiency vs. expressiveness compromise.